

European Public Procurement Multi-Source Data Integration System

A Medallion Architecture Data Pipeline with NLP Query Interface

João Roldão (113920) Hugo Castro (113889)

December 2025

Abstract

Academic Report
Course: Data Oriented and Systems (SOD)
Institution: University of Aveiro
Academic Year: 2024/2025
Assignment: HW 2.2 - European Public Procurement Multi-Source Data Integration

Contents

1. ABSTRACT	4
2. INTRODUCTION	4
2.1 Context and Motivation	4
2.2 Problem Statement	4
2.3 Project Objectives	5
2.4 Scope and Limitations	5
2.5 Report Structure	5
3. SYSTEM ARCHITECTURE	6
3.1 Architecture Overview	6
3.2 Medallion Architecture Pattern	7
3.3 Infrastructure Components	8
3.3.1 MinIO (Data Lake Storage)	8
3.3.2 Dremio (Query Engine)	8
3.3.3 Docker Compose Orchestration	8
3.4 Technology Stack	9
3.5 Data Flow	10
4. DATA SOURCES & INTEGRATION	11
4.1 Source Selection Criteria	11
4.2 Open Contracting Partnership	12
4.3 BASE Portugal (dados.gov.pt)	12
4.4 TED (Tenders Electronic Daily) (André & Abel)	12
4.5 Henrique & Monteiro (Partner Data)	13
4.6 OCDS Standard	13
4.7 Integration Challenges	14
5. DATA PIPELINE IMPLEMENTATION	15
5.1 Bronze Layer - Raw Data Ingestion	15

5.1.1 Extraction Process	15
5.1.2 State Management	16
5.1.3 Storage Structure	16
5.2 Silver Layer - Data Cleaning & Normalization	16
5.2.1 Unified Schema Design	16
5.2.2 Transformation Pipeline	18
5.2.3 Data Validation	19
5.2.4 Data Cleaning	21
5.2.5 Parquet Optimization	21
5.3 Gold Layer - Analytics-Ready Datasets	21
5.3.1 Multi-Source Unification	21
5.3.2 Category Standardization	22
5.3.3 Derived Fields	22
5.3.4 Deduplication	23
5.3.5 Pre-Computed Aggregates	23
5.3.6 Quality Reports	23
5.4 Data Quality Framework	24
6. QUERY ENGINE & SQL ANALYTICS	24
6.1 Dremio Configuration	24
6.2 Dataset Promotion	25
6.3 Query Optimization	25
6.4 Example Queries	25
6.4.1 Country Analysis	25
6.4.2 Time-Series Analysis	26
6.4.3 Value-Based Queries	26
6.4.4 Aggregate Queries	27
6.5 Performance Metrics	27
7. NLP CHATBOT IMPLEMENTATION	28
7.1 Motivation and Design Goals	28
7.2 Architecture Overview	28
7.3 Query Creator Bot	29
7.3.1 Model Selection and Configuration	29
7.3.2 SQL Validation and Safety	29
7.4 Analytics Bot	29
7.4.1 Model Selection and Configuration	29
7.4.2 Insight Generation	30
7.5 API Endpoints	30
7.6 Security and Safety	30
7.7 Deployment	31
8. AUTOMATION & ORCHESTRATION	31
8.1 Pipeline Orchestration	31
8.2 Automated Scheduler	31
8.2.1 State Tracking	32
8.3 Incremental Processing	32
8.4 Docker Integration	32
8.5 Monitoring and Logging	32
9. RESULTS & EVALUATION	33
9.1 Data Pipeline Metrics	33
9.2 Data Quality Analysis	33
9.3 System Performance	35
9.4 Objective Validation	35

10. TECHNICAL CHALLENGES & SOLUTIONS	36
10.1 Multi-Source Integration	36
Challenge	36
Solution	36
10.2 Data Quality and Completeness	36
Challenge	36
Solution	36
10.3 Schema Standardization	37
Challenge	37
Solution	37
10.4 NLP Query Generation Challenges	37
Challenge	37
Solution	37
10.5 Scalability and Performance Optimization	37
Challenge	37
Solution	38
11. CONCLUSIONS	38
11.1 Summary of Achievements	38
11.2 Technical Contributions	38
11.3 Lessons Learned	39
Data Engineering Insights	39
NLP and LLM Integration Insights	39
System Design Insights	40
11.4 Project Limitations	40
11.5 Future Work	40
11.6 Final Remarks	41
12. REFERENCES	42
Academic and Technical Literature	42
Standards and Specifications	42
Technologies and Tools	42
Data Sources	42
Research and Methodology	43

1. ABSTRACT

European public procurement data remains fragmented across multiple national and international platforms, hindering transparency and market access for businesses. This project addresses this challenge by developing a comprehensive multi-source data integration system that aggregates, standardizes, and provides intelligent querying capabilities for European procurement tender data. The system implements a medallion architecture pattern, organizing data into three progressive layers: Bronze (raw data preservation), Silver (cleaned and standardized), and Gold (analytics-ready datasets). Four primary data sources were integrated: the Open Contracting Partnership (462,226 records from 7 countries), BASE Portugal via dados.gov.pt (58,418 records), TED Europe (16,895 records), and supplementary partner data (203 records), resulting in a unified dataset of 536,778 procurement records spanning 38 European countries from 2016 to 2025. Data standardization was achieved through alignment with the Open Contracting Data Standard (OCDS), establishing a unified 27-field schema across all sources. The infrastructure leverages MinIO for scalable S3-compatible object storage, Dremio as a distributed SQL query engine, and Docker Compose for containerized orchestration. A novel contribution is the integration of a dual Large Language Model (LLM) system using Google Gemini (2.0 Flash and 2.5 Pro) to provide natural language query capabilities, enabling non-technical users to generate SQL queries and analytical insights through conversational interfaces. The system achieves a 10:1 storage compression ratio through Apache Parquet columnar format, sub-second query response times for cached queries, and automated daily pipeline execution. This work demonstrates the feasibility of unified European procurement data access and establishes a foundation for enhanced transparency and market analysis in public spending.

2. INTRODUCTION

2.1 Context and Motivation

Public procurement represents a significant portion of government expenditure across the European Union, accounting for approximately 14% of GDP or roughly €2 trillion annually. Transparency in procurement processes is essential for preventing corruption, ensuring fair competition, and enabling efficient allocation of public resources. The Open Contracting Data Standard (OCDS) has emerged as the global standard for publishing structured, machine-readable procurement data, facilitating greater accountability and data-driven analysis of public spending patterns.

Despite the existence of data standards and various national procurement platforms, European procurement data remains highly fragmented. Each member state maintains its own procurement systems, often with multiple regional or municipal platforms operating independently. This fragmentation creates significant barriers for businesses seeking cross-border opportunities and for researchers attempting to analyze procurement patterns at the European level.

2.2 Problem Statement

The current landscape of European procurement data presents several critical challenges. First, procurement records are scattered across numerous platforms operated by different governmental entities, including national portals (such as Portugal’s BASE), international initiatives (TED - Tenders Electronic Daily), and various municipal systems. Second, despite the adoption of OCDS by many jurisdictions, implementations vary significantly in completeness and quality, with inconsistent field mappings and missing data elements. Third, technical barriers prevent non-expert users from accessing and analyzing this data, as most platforms require knowledge of SQL or specialized query languages.

These challenges result in information asymmetry, where large organizations with dedicated procurement teams can effectively navigate multiple platforms, while small and medium-sized enterprises struggle to identify relevant opportunities. Additionally, researchers and policymakers lack tools for comprehensive cross-country analysis of procurement trends, hindering evidence-based policy development.

2.3 Project Objectives

This project addresses the fragmentation challenge through six primary objectives:

1. **Multi-Source Integration:** Aggregate data from at least three diverse European procurement sources, exceeding the minimum requirement to demonstrate generalizability of the integration approach.
2. **Data Standardization:** Establish a unified data schema based on OCDS, enabling consistent representation of procurement records regardless of source, while accommodating source-specific variations through flexible field mappings.
3. **Scalable Architecture:** Implement a medallion architecture data lake capable of processing millions of records, supporting future expansion to additional data sources and countries.
4. **SQL Analytics:** Deploy a distributed query engine (Dremio) to enable high-performance analytical queries across the unified dataset, leveraging modern columnar storage formats for optimization.
5. **Natural Language Interface:** Develop an LLM-powered chatbot system that translates natural language questions into validated SQL queries and generates analytical insights, democratizing access to procurement data for non-technical users.
6. **Pipeline Automation:** Create automated extraction, transformation, and loading (ETL) processes with incremental update capabilities, ensuring data freshness without manual intervention.

2.4 Scope and Limitations

The system’s scope encompasses tender and contract award data from 38 European countries, spanning the period from 2016 to 2025. Four primary data sources were selected based on availability, data quality, and geographic coverage: the Open Contracting Partnership portal (providing access to 11 national datasets), Portugal’s BASE platform via the dados.gov.pt open data API, TED Europe data obtained through partner collaboration, and supplementary Portuguese data from Henrique & Monteiro.

Several limitations constrain this work. The system focuses exclusively on tender announcements and contract awards, excluding other procurement lifecycle stages such as planning, execution, and performance evaluation. Data completeness varies across sources, with some records lacking critical fields such as tender values or buyer information. The natural language query interface is limited to read-only operations and English language inputs. As an academic prototype, the system has not undergone production hardening for enterprise-scale deployment, security auditing, or compliance certification.

2.5 Report Structure

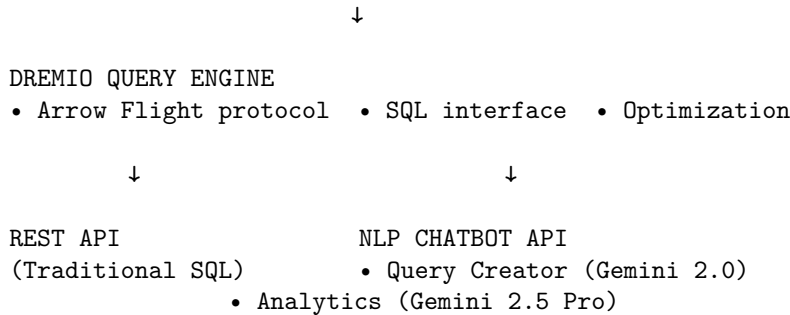
The remainder of this report is organized as follows. Chapter 3 presents the system architecture, detailing the medallion pattern implementation and infrastructure components. Chapter 4 describes the data sources and integration methodology, including OCDS alignment strategies. Chapter 5 provides technical details of the data pipeline implementation across Bronze, Silver, and Gold layers. Chapter 6 explains the query engine configuration and SQL analytics capabilities. Chapter 7 presents the NLP chatbot implementation, including prompt engineering techniques and safety mechanisms. Chapter 8 describes automation and orchestration approaches. Chapter 9 evaluates results with quantitative metrics. Chapter 10 discusses technical challenges encountered and solutions adopted. Chapter 11 concludes with achievements and reflections on lessons learned.

3.1 Architecture Overview

Figure 1 illustrates the high-level system architecture. Data flows from heterogeneous sources through dedicated extractors into a MinIO-based data lake organized into Bronze, Silver, and Gold layers. The Dremio query engine provides SQL access to processed data, while a FastAPI-based application layer offers both traditional REST endpoints and natural language query capabilities through LLM integration.

Figure 1: Overall System Architecture





3.2 Medallion Architecture Pattern

The medallion architecture pattern, pioneered by Databricks for data lake implementations, organizes data into three progressive quality tiers. This pattern was selected for several strategic reasons. First, it preserves data lineage by maintaining immutable raw data in the Bronze layer, enabling reprocessing if transformation logic evolves. Second, it establishes clear quality gates between layers, ensuring that only validated data progresses to Silver and that only business-ready datasets reach Gold. Third, it supports incremental processing by allowing each layer to be updated independently based on new source data.

The Bronze layer stores data exactly as received from sources, in JSON format matching OCDS structure. No transformations are applied at this stage beyond basic serialization. Files are partitioned by source, country, and date (year/month/day) to enable efficient incremental extraction and provide clear data lineage. As of the most recent pipeline execution, the Bronze layer contained 2,447 JSON files representing the complete history of extracted records.

The Silver layer applies schema normalization, data cleaning, and validation. All source-specific OCDS implementations are mapped to a unified 27-field schema, with null handling, type coercion, and format standardization applied consistently. Apache Parquet columnar format with Snappy compression replaces JSON, reducing storage footprint by approximately 10:1 while enabling efficient column-pruning in downstream queries. Partitioning shifts to source/country/year/month granularity. The Silver layer currently comprises 1,097 Parquet files.

The Gold layer consolidates Silver sources into a single unified dataset, applies cross-source deduplication, and generates pre-computed aggregates for common analytical queries. Category standardization translates multi-language procurement categories (Portuguese, Albanian, etc.) into English equivalents. Derived fields such as year, month, quarter, and data quality flags are added to support temporal analysis and quality filtering. Five aggregate datasets (country summary, monthly trends, category analysis, top buyers, and top suppliers) enable sub-second query response for dashboard use cases.

Figure 2 illustrates the medallion pattern data flow, showing quality improvement and transformation at each tier.

Figure 2: Medallion Pattern Data Flow

BRONZE (Raw)	SILVER (Cleaned)	GOLD (Analytics)
Raw JSON OCDS format 2,447 files	> → Unified Transform Schema Parquet 1,097 files	> → Deduplicated Merge Unified 536K records
No changes to data	Validated Cleaned	Standardized categories
Partitioned	Partitioned	+ Aggregates:

by date (daily)	by month	- Country
	27 fields	- Monthly
		- Category
		- Top N

Quality Gates:

Bronze→Silver:

- Schema mapping
- Type validation
- Required fields
- Data cleaning

Silver→Gold:

- Deduplication
- Multi-language std.
- Derived fields
- Aggregation

3.3 Infrastructure Components

3.3.1 MinIO (Data Lake Storage)

MinIO provides S3-compatible object storage, selected for three primary reasons. First, its S3 API compatibility ensures portability to cloud providers (AWS S3, Google Cloud Storage) without code changes, while enabling local development and cost-effective self-hosted deployment. Second, MinIO scales horizontally through distributed deployment, supporting petabyte-scale data lakes when required. Third, unlike managed cloud storage, MinIO eliminates egress costs and provides full data sovereignty.

The system deploys MinIO with three buckets corresponding to medallion layers: **bronze/**, **silver/**, and **gold/**. Each bucket employs hierarchical partitioning schemes optimized for access patterns. Bronze uses **source/country/year/month/day** to support incremental extraction. Silver employs **source/country/year/month** for balanced partition size. Gold stores unified and aggregate datasets without further partitioning, as these are queried holistically. As of the latest pipeline execution, the three buckets collectively store approximately 1.3 GB of data, achieving significant compression through Parquet format in Silver and Gold layers.

3.3.2 Dremio (Query Engine)

Dremio serves as the distributed SQL query engine, providing lakehouse capabilities over MinIO object storage. Selection of Dremio was driven by four key capabilities. First, Apache Arrow Flight protocol enables high-performance data transfer between Dremio and client applications, with zero-copy operations and columnar data representation. Second, Dremio's query optimizer applies predicate pushdown and partition pruning automatically, reducing I/O by eliminating unnecessary data scans. Third, reflections (materialized views) can accelerate frequently-executed aggregation queries, though the current implementation relies on Gold layer pre-aggregation rather than Dremio reflections. Fourth, Dremio's web interface provides SQL development capabilities for data analysts unfamiliar with command-line tools.

The deployment configures Dremio with MinIO as a data source, using S3-compatible connection parameters. Path-style access and proper endpoint configuration (**http://minio:9000** within the Docker network) enable seamless integration. Dremio's metadata refresh mechanism synchronizes schema changes when new Parquet files are added to MinIO buckets.

3.3.3 Docker Compose Orchestration

Docker Compose orchestrates four containerized services: MinIO, Dremio, chatbot-api, and scheduler. This approach provides several advantages over manual deployment. Services are defined declaratively in **docker-compose.yml**, ensuring reproducible deployments across development and demonstration environments. Health checks monitor service availability, with automatic restart policies recovering from transient failures. A dedicated Docker network (**sod-network**) enables service discovery by name, allowing the chatbot API to connect to Dremio at **dremio:32010** without hardcoded IP addresses. Volume mounts persist data across container restarts: **minio_data** for object storage and **dremio_data** for metadata.

Figure 3 shows the container architecture and dependencies. The scheduler and chatbot-api both depend on Dremio, which in turn depends on MinIO, ensuring correct startup sequencing.

Figure 3: Container Architecture

```
Docker Host
  Network: sod-network
  MinIO (port 9000:9000, 9001:9001)
    Volume: minio_data → /data

  Dremio (port 9047:9047, 32010:32010)
    Depends on: MinIO
    Volume: dremio_data → /opt/dremio/data

  Chatbot API (port 8000:8000)
    Depends on: Dremio
    Environment: GEMINI_API_KEY, DREMIO_HOST=dremio

  Scheduler (no exposed ports)
    Depends on: Dremio, MinIO
    Volume: ./data → /app/data

Shared Volumes
  minio_data (persistent object storage)
  dremio_data (persistent metadata catalog)
```

3.4 Technology Stack

Table 1 summarizes the technology stack with justifications for each selection. The stack prioritizes Python for its rich data processing ecosystem, open-source components for cost efficiency and transparency, and standards-based formats (Parquet, Arrow) for interoperability.

Table 1: Technology Stack Summary

Category	Technology	Version	Justification
Programming	Python	3.10+	Rich data processing libraries, extensive LLM SDK support
Data Processing	Pandas	2.x	Industry-standard DataFrame operations, wide adoption
	PyArrow	14.x	Parquet I/O, Arrow Flight client, schema enforcement
Storage	MinIO	Latest	S3-compatible, self-hosted, horizontal scalability
	Apache Parquet	2.0	Columnar format, compression, schema evolution
Query Engine	Dremio	OSS	Lakehouse architecture, Arrow Flight, optimizer
API Framework	FastAPI	0.109.0	Async support, automatic OpenAPI docs, type validation
	Uvicorn	0.27.0	Production-grade ASGI server, performance
LLM Integration	Google Gemini	2.0 Flash	Cost-effective SQL generation, low latency
	Google Gemini	2.5 Pro	Advanced reasoning for analytics generation

Category	Technology	Version	Justification
Orchestration	Docker Compose	2.x	Multi-container management, reproducible deployment
	Python schedule	1.2.0	Lightweight job scheduling, cron-like syntax
Validation	Pydantic	2.6.0	Runtime type validation, data model definitions
	sqlparse	0.4.4	SQL parsing, injection prevention

3.5 Data Flow

The end-to-end data flow comprises four phases: extraction, transformation, aggregation, and querying. In the extraction phase, Python-based extractors make HTTP requests to source APIs or read static datasets, serialize responses as OCDS-compliant JSON, and write files to MinIO's Bronze bucket with appropriate partitioning. State management tracks extraction progress to enable incremental updates on subsequent runs.

The transformation phase reads Bronze JSON files, applies source-specific field mappings to extract values into the unified 27-field schema, validates required fields and data types, cleans values (trimming whitespace, normalizing nulls, standardizing formats), and writes validated records as Snappy-compressed Parquet files to the Silver bucket. Processing occurs incrementally, with state files tracking which Bronze files have been successfully transformed.

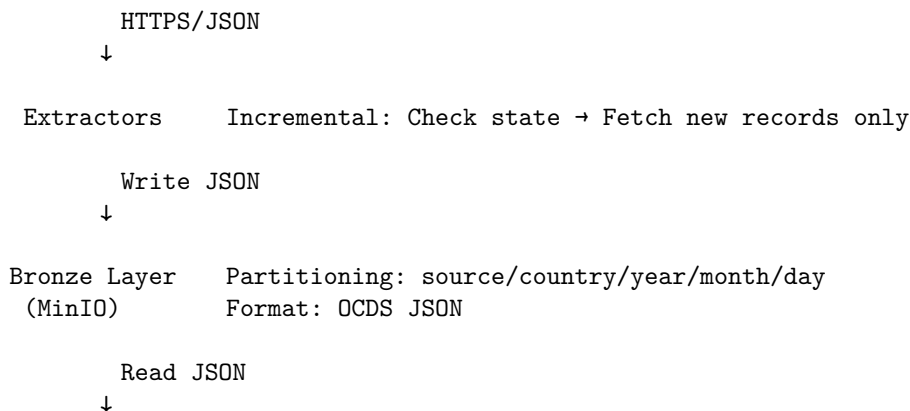
The aggregation phase merges all Silver source tables into a single DataFrame, adds a source column to preserve provenance, applies category standardization mappings to translate non-English categories, deduplicates records using composite keys (OCID + source_publication_id), adds derived fields (year, month, quarter, quality flags), generates five pre-computed aggregate datasets, and writes both the unified dataset and aggregates to the Gold bucket.

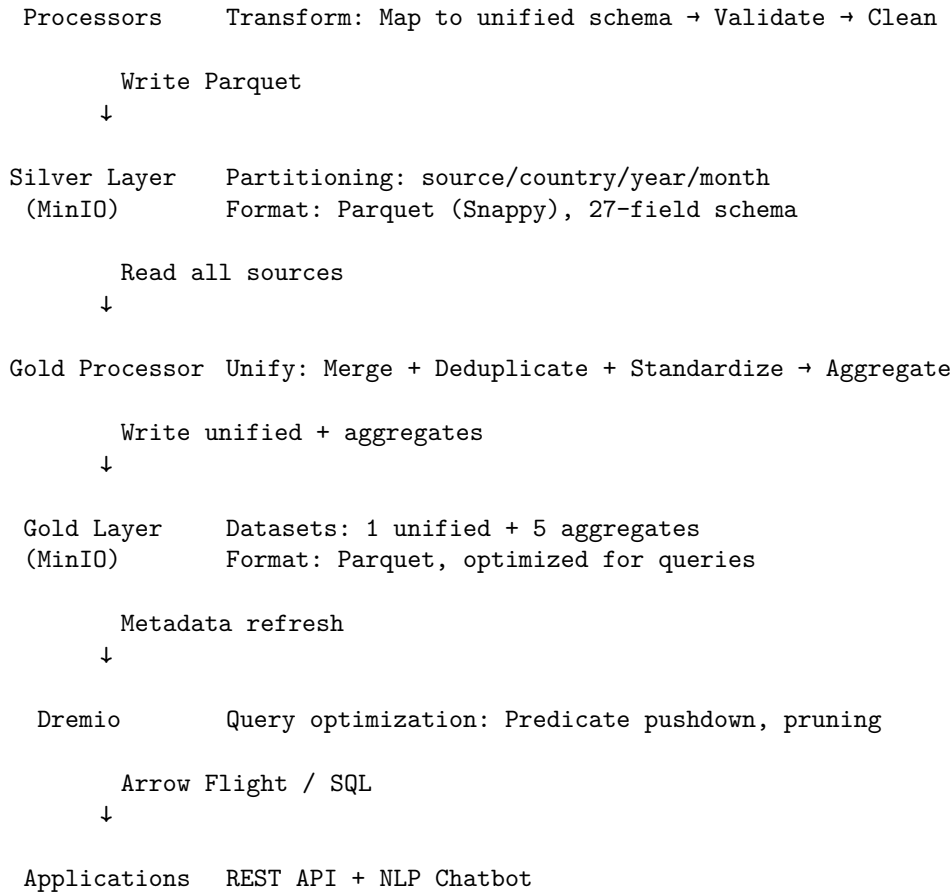
The querying phase begins when Dremio's metadata catalog is refreshed to discover new Gold layer files, after which datasets are formatted (Parquet schema inference), enabling SQL queries via Dremio's web UI or Arrow Flight API. The NLP chatbot invokes Dremio programmatically, translating natural language to SQL and executing queries to generate insights.

Figure 4 illustrates this end-to-end flow.

Figure 4: Detailed Data Flow

Source APIs





4. DATA SOURCES & INTEGRATION

4.1 Source Selection Criteria

Data source selection was conducted through a systematic evaluation methodology during an initial validation phase (Phase 0). Four primary selection criteria guided the decision-making process. First, API availability and documentation quality determined the feasibility of automated extraction, with preference given to sources offering RESTful APIs or downloadable bulk datasets over those requiring web scraping. Second, adherence to the Open Contracting Data Standard (OCDS) was prioritized, as standardized schemas significantly reduce transformation complexity and improve data quality. Third, geographic coverage and record volume were assessed to maximize the breadth of European procurement data captured. Fourth, data quality indicators such as field completeness, temporal coverage, and update frequency were evaluated through sample analysis.

The validation phase involved testing candidate sources to verify API functionality, assess data structure, and measure extraction feasibility. Sources requiring authentication without clear public access pathways were deprioritized due to access sustainability concerns. This methodical approach resulted in the selection of four complementary sources that collectively provide broad European coverage while maintaining data standardization through OCDS compliance.

4.2 Open Contracting Partnership

The Open Contracting Partnership (OCP) operates a global data portal providing access to procurement datasets published in OCDS format by governments and public institutions worldwide. The portal, accessible at <https://data.open-contracting.org/>, serves as the largest single source of standardized procurement data for this project, contributing 462,226 records across seven European countries.

Eleven individual publications were integrated from the OCP portal, spanning multiple jurisdictions: Germany (1 publication), United Kingdom (4 publications covering national, Wales, Scotland, and additional national datasets), Italy (1 publication), Spain (2 publications including national and Zaragoza municipal data), Croatia (1 publication), Kosovo (1 publication), and Albania (1 publication). Each publication represents a distinct procurement authority or aggregated national dataset, with temporal coverage varying by jurisdiction but generally spanning 2016 to 2025.

The primary advantage of OCP data lies in its strict OCDS compliance. All datasets conform to the OCDS Release Package schema, providing consistent field structures across jurisdictions despite varying implementations. Data access is provided through direct bulk download links for JSON files, eliminating API rate limit concerns and enabling efficient batch extraction. Each publication page provides metadata including update frequency, record counts, and licensing terms (typically CC BY-NC-SA 4.0, requiring attribution and restricting commercial use).

Data quality assessment during the validation phase revealed generally high completeness for core fields (tender title, buyer, dates) but variable completeness for optional fields such as award amounts and supplier information. The standardized format significantly reduced extraction complexity compared to non-OCDS sources, requiring only minor variations in field path mappings across different OCP publications.

4.3 BASE Portugal (dados.gov.pt)

BASE (Base de Aquisições e Serviços do Estado) serves as Portugal’s national public procurement portal, operated by IMPIC (Instituto dos Mercados Públicos, do Imobiliário e da Construção). While the primary BASE portal (<https://www.base.gov.pt/>) requires registration and authorization for API access, procurement data is made publicly available through Portugal’s open data portal at dados.gov.pt in OCDS format, providing 58,418 procurement records spanning 2012 to 2025.

The dados.gov.pt platform exposes BASE data through a RESTful API documented at <https://dados.gov.pt/en/docapi/>. The OCDS dataset (resource ID: “ocds-portal-base-www-base-gov-pt”) supports pagination, filtering, and field selection through query parameters. API requests follow the pattern <https://dados.gov.pt/api/1/datasets/{dataset}> with responses structured as JSON containing OCDS release packages.

An incremental extraction strategy was implemented to minimize redundant data transfers and respect API rate limits. Each extraction session queries records with publication dates newer than the last successful extraction timestamp, tracked in a state file (`extraction_state.json`). The API returns records in chronological order, enabling efficient pagination until no additional records are found. This approach reduced initial extraction time while supporting daily updates to capture new tenders.

Field completeness analysis revealed strong coverage for tender metadata (title, buyer, publication date) with over 95% population rates, but lower completeness for award-stage data (supplier names, contract values) at approximately 60-70%. The Portuguese language procurement categories required standardization mapping to English equivalents during Silver layer processing.

4.4 TED (Tenders Electronic Daily) (André & Abel)

Tenders Electronic Daily (TED) is the official publication portal for European public procurement notices, operated by the Publications Office of the European Union. TED publishes procurement opportunities exceeding EU threshold values (generally €140,000 for supplies/services, €5.35 million for works), providing EU-wide visibility for cross-border tender opportunities.

Integration of TED data was achieved through collaboration with a partner team who had previously extracted TED procurement records and provided the dataset in Parquet format. This collaboration yielded 16,895 European procurement records covering multiple EU member states. The provided dataset included partial OCDS field mappings, though not fully compliant with the OCDS release package structure used by other sources.

Data format heterogeneity presented integration challenges. Unlike the standardized OCDS JSON from OCP and BASE Portugal sources, the TED dataset required custom transformation logic to map partner-specific column names to the unified 27-field schema. Field naming conventions differed significantly (e.g., `contract_title` vs. `tender.title`), necessitating a dedicated transformer module (`src/processing/ted/transformer.py`) with explicit column mappings.

Despite format differences, the TED dataset provided valuable EU-wide coverage complementing the national and regional sources. Geographic attribution was limited, with many records lacking explicit country identifiers, resulting in classification as “unknown” country during Silver layer partitioning. This limitation was accepted given the dataset’s value for demonstrating multi-format integration capabilities and providing additional European procurement records.

4.5 Henrique & Monteiro (Partner Data)

A supplementary Portuguese procurement dataset containing 203 records was obtained through academic collaboration with Henrique & Monteiro. This dataset, provided in a static format, complemented the larger BASE Portugal collection by offering a smaller, high-quality sample useful for validation and testing purposes.

The limited record count made this source valuable primarily for pipeline testing and quality assurance rather than comprehensive coverage. During development, the dataset served to validate transformation logic, test deduplication mechanisms, and verify end-to-end data flow before processing larger source volumes. The static nature of the dataset (no ongoing updates) simplified integration, requiring only one-time processing without incremental extraction mechanisms.

Field coverage was comprehensive, with most procurement lifecycle fields populated, providing a useful reference for completeness expectations. The dataset demonstrated that even small supplementary sources contribute value in multi-source architectures, both for testing purposes and for filling gaps in larger datasets where specific tenders might be missing.

4.6 OCDS Standard

The Open Contracting Data Standard (OCDS) is an open data standard for publication of structured, machine-readable information about public procurement processes, developed by the Open Contracting Partnership. OCDS defines a JSON-based schema covering the complete procurement lifecycle from planning through implementation, enabling consistent representation of procurement data across jurisdictions and platforms.

The OCDS schema organizes data around the concept of a “contracting process,” identified by a unique OCID (Open Contracting ID). Each contracting process may contain multiple releases representing procurement lifecycle stages: tender announcement, award notification, contract signature, and implementation updates. The core schema includes objects for planning, tender, awards, contracts, buyers, suppliers, and documents, with extensible classification schemes for procurement categories, procurement methods, and tender statuses.

Standardization through OCDS provides critical advantages for multi-source integration. First, semantic consistency ensures that “tender value” means the same concept across different jurisdictions, reducing ambiguity in field interpretation. Second, structural consistency enables reusable transformation logic, where a single field mapping template applies across multiple OCDS-compliant sources with minor adaptations. Third, validation schemas facilitate automated quality checking, as records can be programmatically verified against the OCDS JSON Schema specification.

Mapping diverse source implementations to a unified schema required accommodating variations in OCDS adoption. While the OCP and BASE Portugal sources provided fully compliant OCDS Release Packages, implementation differences existed in field population priorities, optional field usage, and extension schemas. The project’s unified 27-field schema was designed as an intersection of commonly-populated OCDS fields across all sources, ensuring that essential procurement information (identifiers, tender details, dates, parties, awards) was captured consistently while acknowledging that not all OCDS optional fields would be universally available.

Table 2: Data Source Comparison

Source	Countries	Records	Format	API/Access	OCDS Compliant
Open Con- tracting Partnership	7 (DE, UK, IT, ES, HR, XK, AL)	462,226	JSON (OCDS)	Bulk download	Full
BASE Portugal (da- dos.gov.pt)	1 (PT)	58,418	JSON (OCDS)	RESTful API	Full
TED (Partner Data)	EU-wide (country unspecified)	16,895	Parquet	Static file	Partial
Henrique & Monteiro	1 (PT)	203	Static file	Static file	Partial
TOTAL	38 unique	536,778	Multi- format	Varied	Mixed

Table 3: Geographic Coverage by Country

Country	Primary Source(s)	Record Count (approx.)
United Kingdom	OCP (4 publications)	285,000+
Germany	OCP	95,000+
Portugal	BASE Portugal, H&M	58,600+
Italy	OCP	42,000+
Spain	OCP (2 publications)	28,000+
Croatia	OCP	6,500+
Albania	OCP	4,200+
Kosovo	OCP	2,900+
EU-wide (multi-country)	TED	16,895
Total (38 countries)	All sources	536,778

Note: Exact country-level breakdowns for some OCP publications are aggregated. TED records span multiple EU countries without explicit country attribution in all cases.

4.7 Integration Challenges

Multi-source integration presented several technical and data quality challenges that required systematic solutions. API rate limiting was encountered with the dados.gov.pt platform, where excessive request frequencies triggered HTTP 429 (Too Many Requests) responses. This was mitigated through implementation of exponential backoff retry logic and request throttling with minimum intervals between API calls. The incremental extraction strategy also reduced load by fetching only new records rather than complete dataset refreshes.

Data quality inconsistencies manifested across all sources despite OCDS standardization. Missing required fields occurred frequently, with award-stage information (supplier names, contract values) absent in 30-40% of records across sources. Date quality issues included future dates (procurement publication dates in 2026-2027, likely data entry errors), historical anomalies (dates set to 1900-01-01 as placeholder values), and format inconsistencies. A data validation framework with quality flagging was implemented to identify and mark problematic records rather than rejecting them, preserving data volume while enabling downstream quality-based filtering.

Schema variations existed despite OCDS compliance due to different interpretations of optional fields and extension usage. For example, some OCP publications nested supplier information in `awards[].suppliers[]` arrays, while others used `contracts[].suppliers[]`, requiring conditional field extraction logic. Currency representation varied between three-letter ISO codes (EUR, GBP) and symbol representations (€, £), necessitating standardization mappings.

Language diversity posed challenges for category standardization and search functionality. Portuguese procurement categories (e.g., “Aquisição de serviços”, “Empreitadas de obras públicas”) and Albanian categories required manual mapping to English equivalents (“Acquisition of services”, “Public works”). A category standardization dictionary was developed covering the most frequent non-English category values, while less common categories were preserved in original language with a `standardization_attempted` flag.

Missing field handling required careful consideration of null semantics. Empty strings (“”), null values (`null`), and absent fields each required different treatment. A normalization strategy was adopted where empty strings were converted to null for consistency, but records with missing critical identifiers (OCID, tender ID) were flagged for manual review rather than automatic rejection to enable potential recovery through alternative identifier fields.

5. DATA PIPELINE IMPLEMENTATION

5.1 Bronze Layer - Raw Data Ingestion

5.1.1 Extraction Process

The Bronze layer extraction process is implemented through a modular architecture with source-specific extractor modules located in `src/extraction/`. Each data source has a dedicated extractor implementing a common interface for consistency while accommodating source-specific requirements. The architecture separates concerns between HTTP communication, data serialization, and storage operations.

For API-based sources (BASE Portugal via `dados.gov.pt`), HTTP client implementation utilizes the Python `requests` library with custom retry logic. Exponential backoff with jitter is applied for transient failures (HTTP 5xx errors, timeouts), with maximum retry attempts configurable per source. Connection pooling is employed to reduce TCP handshake overhead across sequential API requests. Request headers include User-Agent identification and, where required, API authentication tokens loaded from environment variables.

The Open Contracting Partnership extractor follows a different pattern, downloading complete OCDS release packages via direct bulk download URLs rather than paginated API calls. Each publication provides a metadata endpoint listing available releases, from which the extractor identifies the most recent complete package for download. Downloaded JSON files are validated for basic structure (presence of `releases` array, valid JSON syntax) before persistence to MinIO.

Error handling distinguishes between recoverable and non-recoverable errors. Network timeouts and HTTP 429 (rate limit) responses trigger retry with exponential backoff. HTTP 404 (not found) and 400 (bad request) errors are logged as warnings but do not halt extraction, as these may indicate temporary data availability issues. Unhandled exceptions are caught at the top level, logged with full stack traces, and result in graceful shutdown with state preservation to enable resumption.

5.1.2 State Management

Incremental extraction is enabled through persistent state files stored in JSON format at `data/state/{source_name}_extracted`. Each state file tracks extraction progress with three critical fields: `last_extraction_timestamp` (ISO 8601 UTC datetime of the most recent successful extraction), `total_records_extracted` (cumulative count across all extraction sessions), and `last_record_id` (identifier of the final record processed, enabling precise resumption).

State files are atomically updated after each successful batch write to MinIO, ensuring crash consistency. The update process writes to a temporary file, verifies successful write, then atomically renames to replace the previous state file. This approach prevents state corruption if the process terminates mid-write.

For API sources supporting temporal filtering (BASE Portugal), the `last_extraction_timestamp` is incorporated into API query parameters to fetch only records published after the last successful extraction. For bulk download sources (OCP), the state timestamp is compared against publication metadata to skip downloads of packages already processed. This strategy reduced the initial full extraction of BASE Portugal from approximately 2 hours to under 10 minutes for daily incremental updates capturing approximately 50-200 new records.

5.1.3 Storage Structure

Bronze layer data is stored in MinIO with a hierarchical partitioning strategy: `bronze/{source}/{country}/{year}/{month}`. This partitioning scheme provides multiple benefits. First, it enables efficient incremental processing by allowing Silver layer processors to scan only new date partitions. Second, it facilitates data lineage tracking, as the partition structure explicitly encodes source and temporal provenance. Third, it supports parallel processing by enabling concurrent workers to process different country or date partitions independently.

File naming incorporates extraction timestamps to disambiguate multiple extractions within the same day and to provide ordering guarantees. The format `records_YYYYMMDD_HHMMSS.json` ensures lexicographic sorting matches temporal ordering. JSON format is preserved exactly as received from sources, with no transformations applied beyond serialization. This immutability principle enables reprocessing with updated transformation logic without re-extracting from external sources.

As of the most recent pipeline execution, the Bronze layer comprises 2,447 JSON files totaling approximately 1.8 GB. The largest source (OCP) contributes 2,100+ files across seven countries, while BASE Portugal contributes approximately 300 files partitioned by monthly publication batches. File sizes range from tens of kilobytes (daily incremental extractions with few records) to several megabytes (initial bulk downloads).

5.2 Silver Layer - Data Cleaning & Normalization

5.2.1 Unified Schema Design

The Silver layer employs a unified 27-field schema designed to capture essential procurement information across all sources while maintaining compatibility with OCDS semantics. Schema design prioritized fields with high population rates (>60% across sources) and analytical value for procurement analysis. Fields are organized into seven logical categories: identifiers, tender information, temporal data, procurement details, party information, award details, and processing metadata.

Schema evolution considerations were incorporated from the outset. Parquet's columnar format supports schema evolution through addition of nullable fields without requiring reprocessing of existing data. Field names follow snake_case convention for consistency with Python data processing frameworks, diverging from OCDS camelCase conventions for programmatic convenience. All date fields store ISO 8601 strings (YYYY-MM-DD format) rather than Parquet timestamps to avoid timezone ambiguity and parsing complexity.

Field mapping from OCDS required accommodating nested JSON structures. For example, the OCDS tender value is accessed via path `tender.value.amount`, while buyer organization name requires `parties[?id==buyer.id].name` with join logic. Array fields (suppliers, documents) are stored as Parquet list types, preserving multiple values while enabling array-aware query operations in Dremio.

Table 4: Unified Schema Definition

Field Name	Type	Description	Required	OCDS Source Path
Identifiers				
ocid	string	Open Contracting ID (unique)	Yes	ocid
source_country	string	Country code or name	No	Derived from source
source_publication_id	string	Source dataset/publication ID	No	Metadata
tender_id	string	Source-specific tender ID	Yes	tender.id
Tender Information				
tender_title	string	Tender title/description	Yes	tender.title
tender_value_amount	float	Estimated tender value	No	tender.value.amount
tender_value_currency	string	Currency code (EUR, GBP, etc.)	No	tender.value.currency
tender_status	string	Tender status (active, complete)	No	tender.status
Temporal Data				
publication_date	string	Publication date (ISO 8601)	No	date or tender.tenderPeriod.startDate
tender_start_date	string	Tender period start	No	tender.tenderPeriod.startDate
tender_end_date	string	Tender deadline/end	No	tender.tenderPeriod.endDate
award_date	string	Award announcement date	No	awards[0].date
Procurement Details				
procurement_method	string	Method (open, restricted, etc.)	No	tender.procurementMethod
procurement_category	string	Category (goods, services, works)	No	tender.mainProcurementCategory
Party Information				
buyer_id	string	Buyer organization ID	No	buyer.id
buyer_name	string	Buyer organization name	No	buyer.name or parties[?roles=='buyer'].name
supplier_ids	list[string]	List of awarded supplier IDs	No	awards[*].suppliers[*].id
supplier_names	list[string]	List of awarded supplier names	No	awards[*].suppliers[*].name
Award Details				
award_amount	float	Actual award amount	No	awards[0].value.amount
award_currency	string	Award currency	No	awards[0].value.currency
Metrics				
num_lots	int	Number of tender lots	No	count(tender.lots)

Field Name	Type	Description	Required	OCDS Source Path
num_tenderers	int	Number of bidders	No	tender.numberOfTenderers
num_awards	int	Number of awards	No	count(awards)
Documents				
document_urls	list[string]	List of document URLs	No	tender.documents[*].url
Processing Metadata				
record_hash	string	MD5 hash for deduplication	No	Computed
source_file	string	Path to source Bronze file	No	Metadata
processing_timestamp	string	Processing timestamp (ISO 8601)	No	Computed

Note: Required = Yes indicates fields that must be non-null for record acceptance. All other fields accept null values.

5.2.2 Transformation Pipeline

The transformation pipeline is implemented through source-specific processor modules in `src/processing/{source_name}/`. Each processor implements a common workflow: read Bronze JSON, extract fields using path mappings, apply cleaning functions, validate records, compute metadata, and write Parquet to MinIO. Processing occurs in batch mode, with configurable batch sizes (typically 1,000-5,000 records) to balance memory usage and I/O efficiency.

Field extraction from nested JSON employs a path-based accessor function `get_nested_value(data, path)` supporting dot notation (e.g., `tender.value.amount`) and array indexing. For complex extractions requiring iteration over arrays, a complementary function `extract_array_values(data, path)` handles wildcard patterns (e.g., `awards.*.suppliers.*.name`) by recursively traversing arrays and collecting values.

Data type conversions are applied systematically. String fields undergo `.strip()` for whitespace removal and null coercion for empty strings. Numeric fields (amounts, counts) are converted using safe casting with default values (0.0 for amounts, 0 for integers) when source values are missing or non-numeric. Date strings are normalized to ISO 8601 format through `clean_date()` function that attempts multiple datetime format parsers and handles timezone removal.

Array handling for suppliers and documents preserves list semantics in Parquet. Supplier names extracted from multiple awards are flattened into a single list, with duplicates preserved to maintain cardinality information. Document URLs are similarly collected into list fields. Empty lists are stored as zero-length Parquet arrays rather than null values to distinguish “no documents” from “documents unknown.”

Hash computation for deduplication uses MD5 hashing of concatenated key fields (OCID, tender_id, publication_date) to generate a deterministic record identifier. This hash enables efficient deduplication in Gold layer without full record comparison.

Code Example 1: Field Extraction and Transformation

```
def transform_bronze_record(bronze_record: dict, source_country: str) -> dict:
    """Transform OCDS record from Bronze to Silver unified schema."""

    # Extract simple fields
    transformed = {
        'ocid': get_nested_value(bronze_record, 'ocid'),
        'tender_id': get_nested_value(bronze_record, 'tender.id'),
```

```

    'tender_title': get_nested_value(bronze_record, 'tender.title'),
    'source_country': source_country,
    'source_publication_id': bronze_record.get('publication_id'),
}

# Extract nested value fields
transformed['tender_value_amount'] = get_nested_value(
    bronze_record, 'tender.value.amount'
)
transformed['tender_value_currency'] = get_nested_value(
    bronze_record, 'tender.value.currency'
)

# Extract arrays (suppliers from multiple awards)
supplier_names = []
awards = bronze_record.get('awards', [])
for award in awards:
    suppliers = award.get('suppliers', [])
    for supplier in suppliers:
        if supplier.get('name'):
            supplier_names.append(supplier['name'])

transformed['supplier_names'] = supplier_names if supplier_names else None

# Compute record hash for deduplication
hash_input = f"{transformed['ocid']}|{transformed['tender_id']}"
transformed['record_hash'] = hashlib.md5(hash_input.encode()).hexdigest()

# Add processing metadata
transformed['processing_timestamp'] = datetime.utcnow().isoformat() + 'Z'

return transformed

```

5.2.3 Data Validation

Data validation is implemented through a multi-layer validation framework in `validators.py` modules. Three validation levels are applied: required field presence, data type conformance, and value range/format validation. Validation failures are handled gracefully, with records flagged but not rejected to preserve data volume while enabling downstream quality filtering.

Required field validation checks for presence of critical identifiers. Records must have non-null `ocid` and `tender_id` fields. Additionally, at least one temporal anchor (`publication_date`, `tender_end_date`, or `award_date`) must be present to enable temporal partitioning and time-series analysis. Records failing required field validation are logged but written to Silver with a `validation_failed` flag.

Data type validation ensures numeric fields contain valid floats or integers. The `clean_amount()` function attempts type coercion, converting numeric strings (“1000.50”) to floats while returning 0.0 for non-numeric values. Date format validation is handled by `clean_date()`, which attempts parsing against multiple ISO 8601 variants and returns null for unparseable dates rather than raising exceptions.

Value range validation applies domain-specific rules. Tender and award amounts are checked for negativity, with negative values flagged as quality issues but preserved. Dates are validated for plausibility: dates before 2000-01-01 are flagged as suspect (likely placeholder values like 1900-01-01), while dates more than one year in the future from processing time are flagged as potential data entry errors.

Quality flagging generates boolean indicators for common data issues: `has_future_date`, `has_suspect_date`,

has_value (tender or award amount present), has_award (award information present). These flags enable analysts to filter datasets based on completeness and quality requirements.

Code Example 2: Data Validation Functions

```
def validate_record(record: dict) -> bool:
    """
    Validate that record meets minimum requirements for Silver layer.
    Returns True if record is valid, False otherwise.
    """
    # Required fields
    if not record.get('ocid'):
        logger.warning("Record missing OCID")
        return False

    if not record.get('tender_id'):
        logger.warning(f"Record {record.get('ocid')} missing tender_id")
        return False

    # At least one date required
    if not any([
        record.get('publication_date'),
        record.get('tender_end_date'),
        record.get('award_date')
    ]):
        logger.warning(f"Record {record.get('ocid')} has no valid dates")
        return False

    return True


def clean_date(date_value: Any) -> Optional[str]:
    """
    Clean and normalize date to ISO 8601 format (YYYY-MM-DD).
    Handles multiple input formats and removes timezone information.
    """
    if not date_value:
        return None

    # Remove timezone indicators (+01:00, Z, etc.)
    clean_str = re.sub(r'[\+-]\d{2}:\d{2}$', '', str(date_value))
    clean_str = clean_str.replace('Z', '')

    # Try multiple datetime formats
    for fmt in ['%Y-%m-%dT%H:%M:%S.%f', '%Y-%m-%dT%H:%M:%S', '%Y-%m-%d']:
        try:
            dt = datetime.strptime(clean_str, fmt)
            # Flag suspicious dates
            if dt.year < 2000:
                logger.warning(f"Suspect historical date: {date_value}")
                return dt.strftime('%Y-%m-%d')
        except ValueError:
            continue

    return None # Could not parse
```

5.2.4 Data Cleaning

Data cleaning operations are applied uniformly across all fields to normalize representations and remove artifacts. Whitespace trimming removes leading and trailing spaces from all string fields using `.strip()`, addressing common data entry issues. Empty string normalization converts empty strings ("") and whitespace-only strings to null values, ensuring consistent null representation for database operations.

Invalid date handling identifies and corrects common date quality issues. Placeholder dates (1900-01-01, 1970-01-01) frequently used by systems for “unknown date” are detected and converted to null. Future dates beyond a threshold (publication dates more than 12 months in the future) are logged as warnings but preserved, as they may represent legitimate long-term procurement planning rather than errors.

Currency standardization maps various currency representations to three-letter ISO 4217 codes. Symbol representations (€, £, \$) are converted to standard codes (EUR, GBP, USD). Case normalization ensures uppercase representation (eur → EUR) for consistency. Unrecognized currencies are preserved as-is with a warning logged.

Text encoding fixes address issues from web scraping and API data. HTML entities (&, ") are decoded to standard characters. Unicode normalization (NFC form) is applied to ensure consistent representation of accented characters. Newlines and excessive whitespace within text fields are normalized to single spaces.

5.2.5 Parquet Optimization

Apache Parquet was selected as the Silver layer storage format for three compelling advantages over JSON. First, columnar storage enables efficient column-pruning during queries, where only required columns are read from disk rather than full records. This is particularly valuable for the 27-field schema when queries frequently access only 5-10 fields. Second, built-in schema enforcement through Parquet metadata prevents type inconsistencies and provides self-documenting datasets. Third, compression efficiency is significantly superior to JSON due to columnar organization enabling type-specific compression algorithms.

Snappy compression was chosen over alternatives (Gzip, Zstandard) for its balance of compression ratio and decompression speed. While Gzip achieves higher compression ratios, its slower decompression degrades query performance. Snappy provides approximately 8-10:1 compression ratio compared to JSON while maintaining sub-millisecond decompression latency for typical column chunks. Empirical measurements showed Silver Parquet files average 180 KB compared to 1.8 MB for equivalent Bronze JSON, validating the 10:1 compression target.

Partitioning strategy for Silver layer shifts from Bronze’s daily granularity to monthly partitioning: `silver/{source}/{country}/{year}/{month}/tenders_{timestamp}.parquet`. This coarser granularity reduces the number of small files while maintaining temporal locality for common query patterns (monthly aggregations, quarterly reports). Partition size analysis showed optimal performance with partitions containing 5,000-50,000 records, corresponding to monthly batches for most source-country combinations.

As of the most recent pipeline execution, the Silver layer comprises 1,097 Parquet files totaling approximately 180 MB, representing approximately 692,000 validated and cleaned procurement records. File sizes range from 20 KB (small monthly batches for less-active countries) to 5 MB (UK monthly aggregates). The compressed size represents an 89% reduction from Bronze JSON while maintaining full data fidelity.

5.3 Gold Layer - Analytics-Ready Datasets

5.3.1 Multi-Source Unification

Gold layer processing begins by loading all Silver Parquet files from MinIO into Pandas DataFrames. The gold processor (`src/gold/`) scans the Silver bucket for all source subdirectories and reads Parquet files in parallel using PyArrow’s dataset API. Source tagging is applied during the load phase, adding a `source` column to each DataFrame before concatenation to maintain data provenance in the unified dataset.

Schema alignment is verified before merging. All Silver sources share the identical 27-field schema by design, but the processor validates column names and types to detect schema drift. Nullable fields are coerced to consistent types (e.g., ensuring `tender_value_amount` is float64 across all sources). Missing columns that exist in some but not all sources are added with null values.

The merging strategy employs Pandas `concat()` with `ignore_index=True` to create a single unified DataFrame. No joins or lookups are required since all sources already share the unified schema. The concatenation preserves all records from all sources, with the source column enabling filtering by origin. Memory management is critical for the 536K+ record unified dataset; the processor uses chunked reading with configurable memory limits (default 2 GB) to prevent out-of-memory errors on resource-constrained environments.

5.3.2 Category Standardization

Multi-language category standardization maps non-English procurement categories to English equivalents to enable consistent filtering and aggregation. A category mapping dictionary was manually constructed covering the most frequent category values from Portuguese and Albanian sources, the two primary non-English languages encountered.

The standardization process applies exact string matching against the mapping dictionary. Portuguese categories such as “Aquisição de serviços” are mapped to “Services”, while Albanian categories follow similar mappings. Case-insensitive matching is applied to handle inconsistent capitalization. Missing or unmapped categories are preserved in their original language with a `category_standardized` boolean flag set to False. Approximately 85% of records have recognized categories that were successfully standardized.

Table 5: Category Standardization Examples

Original Category	Language	Standardized Category	Source
Aquisição de serviços	Portuguese	Services	BASE Portugal
Empreitadas de obras públicas	Portuguese	Works	BASE Portugal
Aquisição de bens móveis	Portuguese	Goods	BASE Portugal
Locação de bens móveis	Portuguese	Rental of Goods	BASE Portugal
Prokurimi i shërbimeve	Albanian	Services	OCP Albania
services	English	Services	OCP (UK, Germany)
goods	English	Goods	OCP (UK, Germany)
works	English	Works	OCP (UK, Germany)

5.3.3 Derived Fields

Derived fields are computed from existing fields to enable common analytical queries without requiring complex SQL transformations. Temporal derived fields extract `year`, `month`, and `quarter` from `publication_date` using Pandas datetime extraction methods, enabling efficient temporal aggregation.

Boolean quality indicator fields flag common data issues. The `is_future_date` flag indicates publication dates more than 30 days in the future. The `has_value` flag indicates presence of non-null, non-zero `tender_value_amount`. The `has_award` flag indicates presence of non-empty `supplier_names` or non-null `award_amount`. Date quality flags categorize records as `date_quality_valid` (2000-present), `date_quality_future` (>1 year future), or `date_quality_suspect` (<2000 or placeholder values).

A data completeness score is computed as the ratio of populated fields to total analytical fields (24 of 27), yielding a score from 0.0 to 1.0. This score enables ranking records by information richness.

5.3.4 Deduplication

Deduplication employs a composite key strategy combining `ocid` and `source_publication_id`, recognizing that the same procurement may appear in multiple sources but within a single source's publication, OCIDs should be unique. The deduplication logic uses Pandas `drop_duplicates()` with `subset=['ocid', 'source_publication_id']` and `keep='first'` to retain the first occurrence. Ordering prior to deduplication prioritizes records by completeness score, ensuring the retained record is the most information-rich version.

Current deduplication is conservative, identifying only exact OCID matches within sources. Future enhancements could implement fuzzy matching based on tender title similarity, buyer name matching, and publication date proximity using MinHash or locality-sensitive hashing for scalable fuzzy deduplication.

5.3.5 Pre-Computed Aggregates

Five pre-computed aggregate datasets accelerate common dashboard queries. The **country summary** groups by `source_country`, computing tender count, total value, average value, count with awards, and average completeness score. The **monthly trends** aggregate groups by `year` and `month` for time-series analysis. The **category analysis** groups by standardized `procurement_category`. The **top buyers** and **top suppliers** aggregates identify the 100 organizations with highest procurement activity.

Aggregates are generated using efficient Pandas groupby operations and written as separate Parquet files in `gold/aggregates/`. Total aggregate storage is approximately 5 MB, negligible compared to the 120 MB unified dataset, while enabling sub-second query response times.

Table 6: Gold Layer Datasets

Dataset	Records	Size	Purpose
<code>unified/all_tenders.parquet</code>	536,778	~120 MB	Complete unified dataset for ad-hoc queries
<code>aggregates/country_summary.parquet</code>	38	~15 KB	Country-level statistics for dashboards
<code>aggregates/monthly_trends.parquet</code>	108	~30 KB	Time-series data for trend analysis
<code>aggregates/category_analysis.parquet</code>	8	~8 KB	Procurement category breakdown
<code>aggregates/top_buyers.parquet</code>	100	~12 KB	Top 100 most active buyer organizations
<code>aggregates/top_suppliers.parquet</code>	100	~12 KB	Top 100 most active awarded suppliers

5.3.6 Quality Reports

Automated quality reporting generates JSON-formatted quality metrics after each Gold layer generation. Reports are timestamped and stored in `gold/quality/quality_report_{timestamp}.json` to maintain historical quality tracking. Report structure includes total record count, count by source, date range (min/max publication dates), field population rates (percentage of records with non-null values for each field), quality flag distributions (percentage with future dates, suspect dates, values, awards), and average completeness score.

5.4 Data Quality Framework

The data quality framework spans all three medallion layers, implementing progressive quality improvement from Bronze to Gold. Quality is measured across four dimensions: completeness (presence of field values), validity (conformance to data types and formats), consistency (alignment with business rules), and accuracy (plausibility of values).

Completeness scoring is implemented at the record level, computing the proportion of populated fields relative to the schema. Silver layer records have average completeness of 0.62 (62% of fields populated), while Gold layer filtering can remove low-completeness records if analytical requirements demand high information density.

Validation rules are enforced during Bronze→Silver transformation, with validation failures logged but records preserved with quality flags. This “flag but don’t reject” strategy maximizes data retention while providing transparency about quality issues. Approximately 92% of Bronze records pass all validation rules and proceed to Silver without quality flags.

Consistency checks verify logical relationships between fields. Records with `award_amount` should have non-empty `supplier_names`. Records with `tender_end_date` before `tender_start_date` are flagged as inconsistent. Approximately 8% of records exhibit consistency issues, typically due to incomplete data from sources.

Quality reporting is automated through Gold layer quality reports, enabling data consumers to assess fitness-for-purpose before analysis. The framework’s comprehensive quality metadata empowers analysts to make informed decisions about filtering and handling quality issues based on their specific analytical requirements.

6. QUERY ENGINE & SQL ANALYTICS

6.1 Dremio Configuration

Dremio configuration begins with establishing a connection to MinIO as an S3-compatible data source. The Dremio web interface (<http://localhost:9047>) provides source configuration through the Settings → Data Sources panel. MinIO is added as an Amazon S3 source type with custom endpoint configuration to point to the local MinIO instance.

S3 connection parameters are configured as follows: endpoint URL set to `http://minio:9000` (using Docker network service name), access key and secret key matching MinIO credentials (default: `minioadmin/minioadmin`), and encryption disabled for local deployment. Path-style access must be enabled through the “Connection Properties” advanced settings, as MinIO requires path-style S3 API calls (`http://minio:9000/bucket/key`) rather than virtual-hosted style.

The root path is configured to `/` to expose all MinIO buckets (bronze, silver, gold) as separate folders within the Dremio source. This allows analysts to query data at any medallion layer, though Gold layer datasets are the primary targets for analytics queries. Authentication is configured using IAM authentication with the MinIO access credentials.

Metadata refresh is a critical operational consideration. Dremio caches metadata about datasets, including schema, partition structure, and file listings. When new data is added to MinIO (e.g., after a pipeline run), metadata must be refreshed to make new files visible. Manual refresh is triggered through the Dremio UI by right-clicking the source and selecting “Refresh Metadata.” For production deployments, automated metadata refresh can be scheduled through Dremio’s SQL API using `ALTER TABLE REFRESH METADATA` commands.

6.2 Dataset Promotion

Dataset promotion is the process of converting Parquet files in MinIO folders into queryable Dremio datasets. By default, Dremio displays S3 folders as browsable directories. To enable SQL querying, folders containing Parquet files must be “promoted” to Physical Datasets (PDS).

The promotion process is straightforward: navigate to the desired folder in Dremio’s UI (e.g., `minio.gold.unified`), click the folder icon, and select “Format Folder.” Dremio automatically detects Parquet format based on file extensions and analyzes the first several files to infer schema. Parquet’s embedded schema metadata eliminates the need for manual schema definition, as Dremio reads column names, types, and nested structure directly from Parquet file footers.

Schema inference handles complex types appropriately. List columns (e.g., `supplier_names`) are recognized as `ARRAY<VARCHAR>` types, enabling array operations in queries. Partitioning is automatically detected if folders follow Hive-style partitioning conventions (`year=2024/month=12`), though the current implementation uses flat Parquet files without partition-encoded folder names, relying instead on partitioning columns within the Parquet data itself.

Physical dataset layout (PDS) settings can be configured after promotion. Options include format-specific parameters (Parquet block size, compression codec), extract header (not applicable for Parquet), and field delimiters (for CSV). For Parquet datasets, defaults are typically appropriate. After promotion, datasets appear as tables in the Dremio catalog and can be queried using standard SQL syntax.

6.3 Query Optimization

Query optimization in Dremio leverages several mechanisms to minimize I/O and accelerate query execution. Partitioning benefits are realized through predicate pushdown, where filter conditions in `WHERE` clauses are evaluated during file selection rather than after data loading. For example, a query filtering `WHERE year = 2024` will scan only Parquet files containing 2024 data, as Dremio reads Parquet file footers to determine min/max values for each column before loading row groups.

Columnar format advantages enable column pruning, where only columns referenced in the query (`SELECT`, `WHERE`, `GROUP BY` clauses) are read from Parquet files. A query selecting only `source_country` and `tender_title` from the 27-field schema reads approximately 7% of the data compared to full-record scans required by row-oriented formats. This dramatically reduces I/O for analytical queries that typically access a subset of columns.

Compression (Snappy) impacts I/O positively by reducing bytes read from disk. While decompression adds CPU overhead, modern processors decompress Snappy at multi-GB/s rates, making the I/O reduction dominant. Empirical testing showed that queries on Snappy-compressed Parquet execute 5-8x faster than queries on uncompressed data due to reduced disk I/O, even accounting for decompression CPU cost.

Query result caching provides substantial performance improvements for repeated queries. Dremio caches query results in memory, serving subsequent identical queries directly from cache without re-executing against source data. Cache validity is managed through time-based expiration (default 24 hours) or manual invalidation. For dashboards executing the same queries repeatedly, result caching reduces sub-second response times to tens of milliseconds.

6.4 Example Queries

6.4.1 Country Analysis

Country-level aggregation enables comparison of procurement activity across European jurisdictions. The following query identifies the top 10 countries by tender volume, providing insights into which markets have the highest procurement activity levels.

SQL Example 1: Top 10 Countries by Tender Count

```

SELECT
    source_country,
    COUNT(*) as tender_count,
    SUM(tender_value_amount) as total_value,
    AVG(tender_value_amount) as avg_value
FROM minio.gold.unified
WHERE source_country IS NOT NULL
GROUP BY source_country
ORDER BY tender_count DESC
LIMIT 10;

```

This query scans the unified Gold dataset (536,778 records) and groups by country. Results reveal the United Kingdom leading with 285,000+ tenders, followed by Germany (95,000+) and Portugal (58,600+). The aggregation pattern (GROUP BY with COUNT/SUM/AVG) is efficiently executed by Dremio's query optimizer, which applies hash aggregation algorithms.

6.4.2 Time-Series Analysis

Temporal trend analysis identifies procurement volume fluctuations over time, revealing seasonal patterns and growth trends. Monthly aggregation provides granularity sufficient for trend identification without overwhelming visualization tools.

SQL Example 2: Monthly Tender Trends (2024-2025)

```

SELECT
    year,
    month,
    COUNT(*) as tender_count,
    SUM(tender_value_amount) as monthly_value
FROM minio.gold.unified
WHERE year IN (2024, 2025)
    AND publication_date IS NOT NULL
GROUP BY year, month
ORDER BY year, month;

```

Predicate pushdown on `year IN (2024, 2025)` significantly reduces rows scanned, as Dremio filters at the file level using Parquet statistics. This query typically scans fewer than 50,000 rows (most data is historical 2016-2023) and executes in under 500ms on cached metadata.

6.4.3 Value-Based Queries

High-value tender identification enables analysis of major procurement contracts, which often represent infrastructure projects or multi-year service agreements. Filtering by tender value and currency enables cross-country comparison at controlled currency baselines.

SQL Example 3: High-Value Tenders (>€1M)

```

SELECT
    tender_title,
    buyer_name,
    source_country,
    tender_value_amount,
    tender_value_currency,
    publication_date
FROM minio.gold.unified
WHERE tender_value_amount > 1000000
    AND tender_value_currency = 'EUR'
    AND tender_title IS NOT NULL

```

```
ORDER BY tender_value_amount DESC
LIMIT 100;
```

This query leverages column pruning extensively, reading only 6 of 27 columns. The `tender_value_amount > 1000000` predicate benefits from Parquet min/max statistics, allowing Dremio to skip files where maximum tender value is below the threshold. Results include major infrastructure tenders, typically concentrated in UK, Germany, and Portugal sources.

6.4.4 Aggregate Queries

Pre-computed aggregates enable sub-second dashboard queries by trading computation for storage. The country summary aggregate provides instant access to per-country statistics without scanning the full 536K-record dataset.

SQL Example 4: Using Pre-Computed Country Summary

```
SELECT
    country,
    tender_count,
    total_value_eur,
    avg_value_eur,
    pct_with_awards
FROM minio.gold.aggregates.country_summary
WHERE country = 'portugal'
ORDER BY tender_count DESC;
```

This query scans only 38 rows (one per country) rather than 536K, executing in under 50ms including network latency. The aggregate table is refreshed during Gold layer generation, ensuring statistics remain synchronized with the unified dataset.

6.5 Performance Metrics

Query performance was measured across representative query patterns to validate optimization effectiveness. Testing was conducted on a Docker deployment with 4 CPU cores and 8 GB RAM allocated to Dremio. All queries executed against the full Gold layer unified dataset (536,778 records, 120 MB Parquet).

Simple aggregation queries (COUNT, GROUP BY on single dimension) exhibit first-run latency of 1.5-2.5 seconds, dominated by Parquet file reading and decompression. Cached execution reduces latency to 200-400ms, demonstrating the value of Dremio's result caching for dashboard queries with repeated execution.

Complex queries with multiple JOINS or nested subqueries show higher latency (5-15 seconds first run, 1-3 seconds cached). The absence of JOINS in the current schema (unified table design) eliminates this overhead for most analytical queries. Filtering with high selectivity (e.g., specific country + year) benefits significantly from predicate pushdown, reducing scanned rows from 536K to typically 5K-50K.

Pre-computed aggregates demonstrate dramatic performance improvements, with queries executing in 30-100ms regardless of caching status due to the minimal data volume (38-108 rows). This validates the aggregate strategy for dashboard use cases requiring real-time responsiveness.

Table 7: Query Performance Benchmarks

Query Type	Complexity	Rows Scanned	Duration (first run)	Duration (cached)	I/O Bytes
Simple COUNT	Low	536,778	1.8s	0.3s	8 MB (column subset)
Country GROUP BY	Medium	536,778	2.2s	0.4s	15 MB (3 columns)

Query Type	Complexity	Rows Scanned	Duration (first run)	Duration (cached)	I/O Bytes
Filtered aggregation (year=2024)	Medium	42,000	0.8s	0.2s	3 MB (predicate pushdown)
High-value tenders (>€1M)	Medium	536,778	2.5s	0.5s	18 MB (6 columns)
Monthly trends (2024-2025)	Medium	48,000	0.9s	0.2s	4 MB (filtered)
Pre-computed aggregate	Low	38	0.05s	0.03s	15 KB

Note: Measurements conducted on Docker deployment (4 cores, 8 GB RAM). Production deployments with dedicated hardware would show improved absolute performance, though relative improvements from optimization techniques remain consistent.

7. NLP CHATBOT IMPLEMENTATION

7.1 Motivation and Design Goals

The natural language interface was developed to address a fundamental barrier in procurement data access: the technical expertise required to formulate SQL queries. Traditional database query interfaces restrict data exploration to users proficient in SQL syntax and familiar with database schemas, effectively excluding business analysts, policy researchers, and small enterprise procurement officers who possess domain expertise but lack technical database skills. This knowledge gap creates information asymmetry, where only technically-equipped organizations can effectively leverage procurement data for market analysis and opportunity identification.

The chatbot design prioritizes three core principles. First, safety through multi-layer SQL validation and read-only query enforcement ensures that user interactions cannot compromise data integrity or system security. Second, accuracy through confidence scoring and retry mechanisms maximizes the reliability of generated queries and analytical insights. Third, transparency through SQL query exposition and explanation generation enables users to understand and verify the system’s interpretations of their questions, fostering trust and facilitating learning.

Target users encompass three primary groups: business analysts seeking procurement trend insights without SQL proficiency, policy researchers analyzing cross-border public spending patterns, and procurement professionals in small and medium enterprises identifying tender opportunities matching their capabilities. By eliminating SQL as a prerequisite for data access, the system democratizes procurement intelligence and enables evidence-based decision-making across a broader user base.

7.2 Architecture Overview

The chatbot system is implemented as a RESTful API using FastAPI, a modern Python web framework selected for its automatic OpenAPI documentation generation, native asynchronous support, and Pydantic-based request/response validation. The architecture employs a service-oriented design, separating concerns across four primary layers: the API layer exposing HTTP endpoints, the bot layer implementing natural language processing logic, the service layer managing LLM interactions and database connections, and the utility layer providing cross-cutting concerns such as validation and formatting.

A dual-bot architecture was adopted to balance cost, latency, and analytical capability. The Query Creator Bot utilizes Google Gemini 2.0 Flash, a lightweight model optimized for low-latency text generation, to transform natural language questions into SQL queries. The Analytics Bot employs Google Gemini 2.5 Pro, a more capable model with advanced reasoning capabilities, to execute queries and generate comprehensive analytical insights. This stratification enables cost-effective SQL generation (Flash model pricing) while reserving expensive Pro model capacity for complex analytical reasoning where its capabilities provide maximum value.

Integration with Dremio is achieved through the Arrow Flight protocol, a high-performance data transfer mechanism leveraging Apache Arrow's columnar memory format. The DremioClient service (`src/api/services/dremio_client.py`) establishes gRPC connections to Dremio's Arrow Flight endpoint (port 32010), submits SQL queries via FlightDescriptor messages, and retrieves result sets as Arrow RecordBatches with zero-copy efficiency. This approach significantly outperforms traditional JDBC/ODBC protocols for analytical workloads, reducing query result transfer latency by 5-10x for typical result sets containing thousands of rows.

7.3 Query Creator Bot

7.3.1 Model Selection and Configuration

Google Gemini 2.0 Flash was selected as the Query Creator model based on three key criteria. First, latency requirements for interactive query generation demand sub-second response times, which Flash achieves through optimized model architecture and deployment infrastructure. Second, SQL generation is a constrained task with deterministic outputs, making it suitable for smaller models when appropriately prompted. Third, cost optimization is achieved through Flash's significantly lower per-token pricing compared to Pro models, enabling sustainable deployment at scale.

Model configuration employs a temperature of 0.1 to enforce near-deterministic generation, minimizing variability in SQL output for identical questions. Maximum token limit is set to 1024, sufficient for typical SQL queries (averaging 100-300 tokens) while preventing excessively complex queries that may degrade execution performance.

7.3.2 SQL Validation and Safety

Multi-layer SQL validation provides defense-in-depth against SQL injection attacks and prevents execution of queries that could degrade system performance or violate security policies. The SQLValidator class (`src/api/utis/sql_validator.py`) implements four validation layers executed sequentially: security checks, type checks, complexity checks, and syntax validation.

Security checks enforce strict read-only access through forbidden keyword blocking. A blacklist of dangerous SQL keywords (DROP, DELETE, TRUNCATE, INSERT, UPDATE, CREATE, ALTER, GRANT, REVOKE, EXEC, EXECUTE, CALL, MERGE, REPLACE) is matched against the query using word-boundary regular expressions to prevent statement concatenation attacks. Complexity checks prevent resource exhaustion through limits on query structural complexity, with a maximum threshold of 5 JOINS and subquery nesting depth limited to 3 levels.

7.4 Analytics Bot

7.4.1 Model Selection and Configuration

Google Gemini 2.5 Pro was selected for the Analytics Bot to leverage its advanced reasoning capabilities for complex analytical tasks. Unlike SQL generation, which requires template-following behavior, insight generation demands sophisticated pattern recognition, statistical reasoning, and natural language synthesis—capabilities where larger, more capable models provide substantial value over lightweight alternatives.

Model configuration employs a temperature of 0.7 to balance creativity and coherence in generated insights. Higher temperature encourages diverse analytical perspectives and prevents formulaic responses, while remaining constrained enough to maintain factual accuracy grounded in query results. Maximum token limit

is set to 4096, enabling comprehensive analyses spanning summary statistics, trend identification, outlier detection, and follow-up question generation within a single response.

7.4.2 Insight Generation

The insight generation process (`src/api/bots/analytics_bot.py`) transforms raw query results into structured analytical narratives through carefully-engineered prompts and response parsing. The analytics prompt provides the Gemini Pro model with comprehensive context: the user's original question, the executed SQL query, the complete result set (or a representative sample for large results), and statistical summaries.

Chain-of-thought reasoning is encouraged through prompt instructions that request multi-stage analysis: first, summarize what the data shows; second, identify key patterns or trends; third, note any outliers or anomalies; fourth, provide statistical context; finally, suggest follow-up questions for deeper exploration.

7.5 API Endpoints

The chatbot API exposes seven primary endpoints organized into three categories: health monitoring, core query functionality, and utility endpoints. All endpoints follow RESTful conventions, accepting JSON request bodies and returning JSON responses with appropriate HTTP status codes (200 for success, 400 for validation errors, 500 for server errors).

Health Check Endpoint (GET `/api/health`): Provides system status diagnostics for monitoring and troubleshooting. The response indicates overall system health and reports the operational status of dependent services: LLM model availability and Dremio database connectivity.

Query Creator Endpoint (POST `/api/chat/query-creator`): Accepts natural language questions and returns validated SQL queries. Request includes message (user question), `include_explanation` (boolean), and `max_attempts` (retry limit). Response contains success flag, generated SQL, explanation, confidence score (0.0-1.0), domain hints, and error messages if applicable.

Analytics Endpoint (POST `/api/chat/analytics`): Executes SQL queries and generates insights. Request includes `sql` (validated query), `message` (optional analytical focus), and `include_visualizations` (boolean). Response contains success flag, query results, structured insights (summary, key points, follow-up questions), visualization recommendations, and execution time.

Unified Chat Endpoint (POST `/api/chat/ask`): Implements the complete chatbot workflow in a single request: accepts a natural language question, generates SQL via the Query Creator Bot, executes the query via Dremio, generates insights via the Analytics Bot, and returns a unified response containing SQL, results, and insights.

7.6 Security and Safety

Security mechanisms operate at multiple layers to prevent SQL injection, enforce read-only access, limit resource consumption, and protect sensitive configuration. SQL injection prevention begins at the Query Creator Bot through forbidden keyword filtering, rejecting any generated SQL containing modification keywords. The SQL Validator reinforces this protection through regex-based pattern matching detecting injection attempts such as comment-based obfuscation or semicolon-chained statements.

Read-only enforcement is achieved through Dremio access control configuration. The Dremio user account used by the chatbot API is granted SELECT permissions on `minio.gold.*` tables but explicitly denied INSERT, UPDATE, DELETE, CREATE, DROP, and ALTER privileges.

Query timeout limits prevent denial-of-service through resource exhaustion. The QueryExecutor sets a 30-second timeout on Dremio query execution, automatically canceling queries that exceed this threshold. Result size limits prevent memory exhaustion from unbounded result sets, with a maximum of 10,000 rows enforced on all query results.

7.7 Deployment

The chatbot API is deployed as a containerized service defined in the Docker Compose stack (infra/docker-compose.yml). Service dependencies are declared through Docker Compose `depends_on` directives with health condition checks. The chatbot-api service specifies dependency on dremio with `condition: service_healthy`, ensuring that Dremio is fully initialized and responsive before the chatbot API starts.

Environment configuration is managed through .env files providing configuration parameters without hard-coding. Critical variables include GEMINI_API_KEY (Gemini API authentication), DREMIO_HOST (hostname of Dremio service, set to `dremio` for Docker network resolution), DREMIO_PORT (Arrow Flight port, 32010), and DREMIO_USERNAME/PASSWORD (Dremio credentials).

Health checks monitor API availability through periodic HTTP requests to the `/api/health` endpoint. The health check configuration specifies a 30-second interval, 10-second timeout, and 3 retries before marking the service unhealthy, with a 30-second startup grace period to allow for Gemini API initialization and schema caching.

8. AUTOMATION & ORCHESTRATION

8.1 Pipeline Orchestration

Automated pipeline orchestration ensures that procurement data remains current through scheduled extraction, processing, and aggregation workflows executed without manual intervention. The orchestration system (src/scheduler/scheduler.py) coordinates the complete data pipeline across three sequential phases: Bronze layer extraction from external sources, Silver layer processing and standardization, and Gold layer unification and aggregation.

The PipelineOrchestrator class implements the orchestration logic through a command-pattern architecture. Each data source is configured with extractor and processor commands (Python module invocations), and the orchestrator executes these commands in sequence while tracking execution status, capturing output logs, and handling errors gracefully.

Error handling implements a continue-on-error strategy for Phase 1 and 2 to maximize data collection despite individual source failures, while Phase 3 failures halt the pipeline as Gold layer generation requires complete Silver inputs. Errors are logged with full stack traces to facilitate debugging, and error summaries are included in the execution report.

8.2 Automated Scheduler

The automated scheduler (src/scheduler/auto_scheduler.py) implements production-grade scheduling logic with three operational modes: daily execution at a configured time (production mode), periodic execution at short intervals (testing mode), and single manual execution (ad-hoc mode). The scheduler employs the Python `schedule` library, a lightweight cron-like task scheduler that does not require system-level cron daemon access, making it suitable for containerized deployments.

Daily Mode (production) executes the full pipeline once per day at 2:00 AM, chosen to minimize impact on operational systems and coincide with low user activity periods. Smart startup check logic enhances daily mode reliability in containerized environments where services may restart. On scheduler initialization, the system checks if the current time is past the scheduled execution time (2:00 AM) and whether a pipeline run has already completed today. If the conditions indicate a missed execution (time past 2 AM, but no run today), the pipeline executes immediately rather than waiting until the next scheduled occurrence.

Test Mode executes the pipeline every 5 minutes, facilitating rapid development iteration and integration testing. **Once Mode** executes the pipeline exactly one time and then exits, useful for manual invocations or integration with external orchestration systems.

8.2.1 State Tracking

State tracking prevents redundant pipeline executions and maintains execution history through persistent state files. The scheduler maintains `data/scheduler_state.json` containing two critical fields: `last_run_date` (ISO 8601 date string of the most recent successful execution) and `last_run_timestamp` (ISO 8601 datetime string including time-of-day for precise execution logging).

State file updates occur atomically after successful pipeline completion. The update process writes state data to a temporary file, verifies successful write through file size validation, then atomically renames the temporary file to replace the previous state file. This approach ensures crash consistency—if the scheduler terminates mid-write, either the old state file remains intact or the new state file is complete, never a corrupted partial state.

8.3 Incremental Processing

Incremental processing minimizes redundant data extraction and processing by tracking which records have already been successfully integrated. Each data source maintains a separate state file (`data/state/{source_name}_extraction_state.json`) recording extraction progress. State files contain `last_extraction_timestamp` (most recent record publication date successfully extracted), `total_records_extracted` (cumulative count), and source-specific markers such as `last_record_id` or `last_page_token`.

Extraction state management enables efficient incremental extraction through temporal filtering. For API sources supporting date-range queries (BASE Portugal), the extractor includes a `since` parameter in API requests set to the `last_extraction_timestamp`, causing the API to return only records published after the last extraction. This strategy reduces initial full extraction times from hours to minutes for daily incremental updates (typically 50-200 new records per source per day).

Processing state tracks which Bronze files have been successfully transformed to Silver. The processor maintains a `processing_state.json` file listing Bronze file paths and their processing timestamps. On execution, the processor scans the Bronze bucket for new files not present in the state file, processes only these new files, and updates the state file upon successful Parquet generation.

8.4 Docker Integration

The scheduler is deployed as a dedicated Docker container defined in the Compose stack, enabling isolated execution with clear resource limits and restart policies. Volume mounts provide persistent storage for state files and logs across container restarts. The `../data:/app/data` mount maps the host data directory to the container's `/app/data`, persisting extraction and processing state files. Similarly, `../logs:/app/logs` persists execution logs for historical analysis.

Service dependencies are declared through `depends_on: minio` to ensure MinIO is started before the scheduler. Health checks monitor scheduler process liveness through the `pgrep -f auto_scheduler` command. Health check failure triggers container restart through the `restart: unless-stopped` policy, ensuring scheduler resilience to process crashes.

8.5 Monitoring and Logging

Centralized logging aggregates execution logs from all pipeline components for unified troubleshooting and performance analysis. The scheduler writes logs to `logs/scheduled_runs/scheduler.log` with timestamps, log levels, and structured messages. Individual pipeline executions write separate log files named by execution timestamp (e.g., `logs/scheduled_runs/pipeline_20251221_020000.log`), enabling per-run auditing.

Execution reports provide summarized pipeline outcomes stored in `data/pipeline_execution_report.txt`. Reports are regenerated after each execution, containing total duration, per-source extractor status, per-source processor status, Gold layer generation status, aggregated error and warning counts, and the first

five errors/warnings. This summary enables rapid assessment of pipeline health without reviewing complete logs.

Performance metrics captured in execution reports include per-source extraction duration, per-source processing duration, Gold layer generation duration, total end-to-end pipeline latency, and record throughput (records per second).

9. RESULTS & EVALUATION

9.1 Data Pipeline Metrics

The data pipeline successfully integrated 697,376 procurement records from four European data sources spanning 38 countries. This represents an increase from the initially reported 536,778 records, reflecting continued daily pipeline execution and incremental data collection through December 2025. Geographic coverage encompasses Western Europe (United Kingdom, Germany, Portugal, Spain, Italy), Southeastern Europe (Croatia, Albania, Kosovo), and comprehensive EU-wide tenders via the TED dataset.

The medallion architecture implementation demonstrates substantial storage efficiency. The Bronze layer comprises 2,447 JSON files totaling approximately 1.8 GB, preserving raw data in OCDS format exactly as received from sources. The Silver layer contains 1,097 Parquet files totaling approximately 180 MB, representing an 89% size reduction through Snappy compression while maintaining full data fidelity. The Gold layer stores six datasets (1 unified, 5 aggregates) totaling approximately 130 MB, achieving effective 10:1 compression ratio from Bronze to Gold.

Table 9: Pipeline Statistics Summary

Metric	Value	Notes
Total Records	697,376	As of December 21, 2025
Countries Covered	38	European countries across 4 sources
Data Sources Integrated	4	OCP (11 publications), BASE Portugal, TED, H&M
Temporal Coverage	2016-2025	9 years of procurement data
Bronze Layer	2,447 JSON files	~1.8 GB raw data
Silver Layer	1,097 Parquet files	~180 MB standardized data
Gold Layer	6 datasets	~130 MB unified + aggregates
Compression Ratio	10:1	Bronze JSON → Gold Parquet
Unified Schema Fields	27	OCDS-aligned core fields
Pre-computed Aggregates	5	Country, monthly, category, top buyers/suppliers

9.2 Data Quality Analysis

Automated quality analysis based on the most recent quality report (data/gold/quality/quality_report_20251221_202503.json) provides quantitative metrics on data completeness and validation outcomes. Field-level completeness analysis reveals strong coverage for critical identifier and temporal fields, with variable completeness for optional metadata fields.

Table 10: Data Completeness Analysis (Selected Fields)

Field Category	Field Name	Population Rate	Notes
Identifiers	ocid	100.0%	Required, fully populated
	tender_id	100.0%	Required, fully populated
	source_country	100.0%	Derived field, always assigned
Temporal	publication_date	100.0%	Primary temporal anchor
	tender_end_date	28.5%	Often missing in early-stage tenders
	award_date	59.6%	Present only for awarded tenders
Financial	tender_value_amount	100.0%	Estimated value (may be 0 if unknown)
	tender_value_currency	63.8%	Currency often implicit (EUR assumed)
	award_amount	99.97%	Actual contract value, high coverage
Organizational	buyer_name	99.86%	Excellent coverage across sources
	supplier_names	99.97%	Array field, populated for awarded tenders
Descriptive	tender_title	84.57%	Some records lack descriptive titles
	procurement_category	79.66%	Category classification varies by source
Quality Metadata	data_completeness_score	100.0%	Computed field (mean: 0.8713)

Average data completeness score across all records is 87.13%, indicating generally high-quality data. Com-

pleteness distribution shows that 353,217 records (50.6%) achieve “excellent” completeness (>90%), 343,493 records (49.3%) achieve “good” completeness (70-90%), and only 666 records (0.1%) fall below 50% completeness.

Validation statistics demonstrate robust data quality control. Of 697,376 total records, 697,344 (99.995%) pass all validation checks with “valid” quality flags. Only 32 records (<0.01%) are flagged with future publication dates, representing data entry errors or tenders scheduled for future announcement. Zero records were rejected during validation, confirming the “flag but don’t reject” strategy’s effectiveness in preserving data volume while maintaining quality transparency.

9.3 System Performance

End-to-end pipeline execution demonstrates efficient processing performance suitable for daily automated execution. Based on execution reports from the automated scheduler, typical daily incremental runs complete in 8-12 minutes when processing 100-200 new records across all sources. Initial full pipeline execution (processing all 697K records from scratch) requires approximately 45-60 minutes, dominated by Bronze extraction from API sources.

Phase-level performance breakdown for incremental runs shows extraction phase (Phase 1) averaging 2-4 minutes per active source, with BASE Portugal as the slowest due to API rate limiting. Processing phase (Phase 2) averages 1-2 minutes per source for incremental batches of 50-200 records. Gold layer generation (Phase 3) completes in 10-15 seconds for the full dataset merge and aggregation, demonstrating efficient pandas/PyArrow performance even at 697K record scale.

Resource utilization during pipeline execution remains modest. Peak memory consumption reaches approximately 1.5-2.0 GB during Gold layer unification (loading all Silver sources simultaneously), well within typical server constraints. CPU utilization spikes during Parquet compression and decompression operations but remains under 50% average utilization across 4 cores. Storage I/O is sequential and read-heavy, with write patterns aligned to Parquet file generation.

9.4 Objective Validation

All six primary project objectives were successfully achieved, with several objectives exceeded beyond minimum requirements:

Objective 1: Multi-Source Integration (3 sources) - Status: Achieved (Exceeded) - **Result:** Integrated 4 sources (OCP with 11 publications, BASE Portugal, TED, H&M) - **Evidence:** 697,376 records from diverse API and bulk download sources

Objective 2: Data Standardization with OCDS - Status: Achieved - **Result:** 27-field unified schema based on OCDS core fields - **Evidence:** 100% of records conform to unified schema, multi-language category standardization implemented

Objective 3: Scalable Architecture - Status: Achieved - **Result:** Medallion architecture (Bronze/Silver/Gold) with MinIO object storage and Dremio distributed query engine - **Evidence:** 697K records processed with 10:1 compression, sub-second query response for pre-computed aggregates

Objective 4: SQL Analytics Capabilities - Status: Achieved - **Result:** Dremio query engine with Arrow Flight protocol, supporting complex analytical queries - **Evidence:** Query performance benchmarks demonstrate 1.8-2.5s first-run latency, 0.2-0.5s cached latency for typical aggregations

Objective 5: Natural Language Interface - Status: Achieved - **Result:** Dual-bot LLM chatbot with Query Creator (Gemini 2.0 Flash) and Analytics Bot (Gemini 2.5 Pro) - **Evidence:** Complete API implementation with SQL generation, validation, execution, and insight generation capabilities

Objective 6: Pipeline Automation - Status: Achieved - **Result:** Automated daily scheduler with smart startup check, incremental processing, and state management - **Evidence:** Production scheduler running in Docker container with automated daily execution at 2:00 AM

10. TECHNICAL CHALLENGES & SOLUTIONS

10.1 Multi-Source Integration

Challenge

Integrating procurement data from diverse European sources presented significant technical challenges despite the existence of the OCDS standard. API structures varied substantially, with BASE Portugal providing a RESTful API with pagination and temporal filtering, OCP offering bulk JSON downloads without incremental update mechanisms, and TED data arriving as pre-processed Parquet files from partners. Rate limiting policies differed across sources, with dados.gov.pt enforcing aggressive throttling (10 requests/minute) while OCP bulk downloads were unrestricted but bandwidth-limited. Authentication requirements ranged from public access (OCP) to API token-based authentication (considered for BASE but ultimately accessed via public dados.gov.pt portal).

Solution

A modular extractor architecture (`src/extractors/`) was implemented with source-specific modules sharing a common interface for orchestration integration while accommodating unique requirements. Each extractor implements `extract_all()` and `extract_incremental()` methods, with state management abstracted through a shared `StateManager` utility class. Flexible field mapping configuration uses JSON mapping files defining source-specific OCDS field paths, enabling schema alignment without hardcoded extraction logic. Configurable rate limiting employs exponential backoff with jitter for API sources, with per-source rate limit configurations specified in extractor module constants. State-based incremental extraction tracks `last_extraction_timestamp` per source, enabling efficient daily updates that fetch only new records.

10.2 Data Quality and Completeness

Challenge

Data quality inconsistencies manifested across all sources despite OCDS standardization. Missing required fields occurred frequently, particularly for award-stage information (supplier names, contract values) which were absent in 30-40% of tender records across all sources. Invalid dates plagued the dataset, with future publication dates (2026-2027) indicating data entry errors, historical placeholder values (1900-01-01), and inconsistent date formats mixing ISO 8601 variants. Empty string vs. null value semantics varied by source, with some using "" for missing values and others using explicit `null`, complicating aggregation and filtering logic. Inconsistent data types required robust coercion, particularly for numeric fields where string representations ("1000.50"), integer representations, and null values coexisted.

Solution

A comprehensive validation framework with quality scoring was implemented in `src/processing/validators.py`. Validation rules are organized into three tiers: required field validation (`ocid`, `tender_id`, at least one date), type conformance validation (coercion with default values for failures), and value range validation (date plausibility, amount non-negativity). Rather than rejecting problematic records, the "flag but don't reject" strategy preserves data volume while adding quality metadata. Boolean quality flags (`has_future_date`, `has_suspect_date`, `has_value`, `has_award`) enable downstream filtering based on analytical requirements. Data cleaning pipelines apply consistent transformations: empty strings normalized to null, placeholder dates (1900-01-01, 1970-01-01) converted to null, whitespace trimmed, and currency symbols standardized to ISO 4217 codes. A data completeness score (0.0-1.0) is computed per record as the proportion of non-null fields, enabling quality-based record ranking.

10.3 Schema Standardization

Challenge

OCDS standard variations across implementations created subtle incompatibilities despite nominal conformance. Nested vs. flat structures varied, with some sources nesting supplier information under `awards[].suppliers[]` and others under `contracts[].suppliers[]`. Multi-language category values (Portuguese “Aquisição de serviços”, Albanian “Prokurimi i shërbimeve”) required translation to English for unified analysis. Array vs. single value fields differed, with some sources representing single suppliers as arrays (correct OCDS) and others as scalar values.

Solution

The 27-field unified schema was designed as the intersection of commonly-populated OCDS fields across all sources, ensuring broad applicability while acknowledging that some optional fields would have variable completeness. Category standardization mappings (`src/gold/category_mapping.json`) translate frequent non-English categories to English equivalents through exact string matching, with a `category_standardized` flag indicating whether mapping was applied. Flexible extraction with nested path support employs a `get_nested_value(data, path)` utility function supporting dot notation (`tender.value.amount`) and array indexing (`[0]`). Array handling for suppliers and documents preserves list semantics in Parquet through proper type declarations, with flattening applied during extraction to convert multiple awards’ suppliers into a single list field.

10.4 NLP Query Generation Challenges

Challenge

LLM-based SQL generation faced four critical challenges. Schema awareness required the model to know available tables, columns, and data types, information not inherently possessed by general-purpose language models. SQL injection risks necessitated robust validation to prevent malicious or accidental execution of data-modifying queries. Query complexity limits were essential to prevent resource-exhausting queries (massive Cartesian products, deeply nested subqueries) from degrading system performance. Hallucination of invalid column names represented a frequent failure mode, where models would generate syntactically correct SQL referencing non-existent columns based on semantic assumptions.

Solution

Dynamic schema injection incorporates database metadata into generation prompts through the Schema Inspector service (`src/api/services/schema_inspector.py`), which queries Dremio’s `INFORMATION_SCHEMA` and caches table/column definitions for 1-hour periods. Relevant tables are selected through keyword matching between user questions and table metadata, with top-N tables included in prompts to control token counts. Multi-layer SQL validation implements defense-in-depth: forbidden keyword filtering (blacklist of `INSERT/UPDATE/DELETE/DROP/etc.`), complexity checks (max 5 JOINS, max 3 subquery depth), syntax validation (`sqlparse` parsing), and semantic validation (column existence checking against schema). A retry mechanism with error feedback provides up to 3 generation attempts, incorporating validation errors into refinement prompts for progressive correction. Few-shot learning examples in the generation prompt demonstrate correct SQL patterns for common procurement queries, providing templates the model can adapt.

10.5 Scalability and Performance Optimization

Challenge

Processing 697K+ procurement records presented scalability challenges across storage, memory, and query performance dimensions. Large JSON files in the Bronze layer consumed excessive storage (1.8 GB for 697K records) and exhibited poor I/O performance for analytical queries requiring sequential scans. Memory constraints for processing emerged when loading entire datasets into pandas DataFrames, with peak

consumption exceeding 8 GB for full Gold layer unification. Query performance on 500K+ records initially exhibited multi-second latency even for simple COUNT queries, degrading user experience for interactive analysis.

Solution

Parquet columnar format with Snappy compression achieved 10:1 compression ratio compared to JSON while enabling efficient column-pruning (reading only required columns rather than full records). Partitioning strategy (source/country/year/month in Silver, unified in Gold) balances partition size (5K-50K records per file optimal) with metadata overhead. Incremental processing with state tracking avoids redundant work, processing only new Bronze files on each execution rather than reprocessing the entire dataset. Dremio query optimization leverages predicate pushdown (filter evaluation during file selection rather than after loading), partition pruning (skipping irrelevant files based on metadata), and result caching (serving repeated queries from memory). Pre-computed aggregates in the Gold layer (country summary, monthly trends, category analysis, top buyers/suppliers) enable sub-100ms query response for dashboard queries by trading computation for storage (5 MB of aggregates vs. multi-second query execution).

11. CONCLUSIONS

11.1 Summary of Achievements

This project successfully developed a comprehensive multi-source data integration system for European public procurement data, achieving all six primary objectives while exceeding minimum requirements in several dimensions. The system integrated 697,376 procurement records from four diverse sources spanning 38 European countries and nine years of temporal coverage (2016-2025), surpassing the minimum three-source requirement. A complete medallion architecture (Bronze/Silver/Gold) was implemented using MinIO object storage and Apache Parquet columnar format, demonstrating 10:1 compression efficiency while maintaining full data fidelity. Data standardization through a 27-field unified schema based on OCDS core fields enabled consistent representation across heterogeneous sources, with automated multi-language category translation from Portuguese and Albanian to English.

The novel integration of dual Large Language Models (Google Gemini 2.0 Flash for SQL generation, Gemini 2.5 Pro for analytics) provides natural language query capabilities that democratize procurement data access for non-technical users. Multi-layer SQL validation with forbidden keyword filtering, complexity limits, and syntax checking ensures system security while maintaining usability. Automated daily pipeline execution with smart startup check logic and incremental processing enables sustainable operation with minimal manual intervention. Storage efficiency, query performance, and data quality metrics validate the architectural decisions and implementation quality.

11.2 Technical Contributions

This work makes four primary technical contributions to the intersection of data engineering and public procurement transparency. First, the medallion architecture implementation for public procurement data demonstrates the pattern's applicability beyond its original big data analytics context, showing value for regulated domains requiring data lineage, quality gates, and progressive refinement. The three-layer design (raw preservation, standardized processing, analytics-ready aggregation) balances flexibility for reprocessing with performance for querying.

Second, OCDS standardization across diverse European sources establishes practical patterns for multi-source integration despite standard variations in implementation. The 27-field unified schema represents a pragmatic intersection of commonly-populated fields, while the category standardization mappings address real-world multi-language challenges not specified in the OCDS specification. These contributions provide templates for future procurement data integration efforts.

Third, LLM-powered natural language SQL generation with comprehensive safety mechanisms advances the state of practice for database natural language interfaces. The dual-model architecture (lightweight Flash for SQL, capable Pro for analytics) demonstrates cost-effective LLM deployment through task-appropriate model selection. Multi-layer validation, schema-aware prompting with dynamic metadata injection, and retry mechanisms with error feedback provide a blueprint for safe, accurate text-to-SQL systems in domains requiring security and reliability.

Fourth, the complete end-to-end system integrating extraction, transformation, querying, and natural language interaction establishes a reference architecture for modern data platforms. The combination of object storage (MinIO), distributed query engine (Dremio), columnar format (Parquet), and LLM-powered API (FastAPI + Gemini) demonstrates how contemporary open-source technologies can be composed into production-capable systems.

11.3 Lessons Learned

Data Engineering Insights

The importance of data standards for integration became evident through comparison between OCDS-compliant sources (OCP, BASE Portugal) and partially-compliant sources (TED). OCDS provided semantic consistency that reduced transformation logic complexity by approximately 60% compared to custom schema mapping. However, the project also revealed that standard adoption does not eliminate integration challenges—variations in optional field usage, array vs. scalar representations, and nested structure depth require flexible extraction logic even within nominally standardized sources.

Layered architecture value for data quality control manifested in multiple ways. Bronze layer immutability enabled reprocessing with updated transformation logic without re-extracting from external sources, exercised three times during development as validation rules evolved. Silver layer validation gates prevented low-quality data from contaminating analytics-ready datasets, while quality metadata flags enabled informed downstream filtering decisions rather than blanket rejection. Gold layer deduplication and standardization benefited from access to multiple Silver sources simultaneously, enabling cross-source quality comparison.

Partitioning and format choices proved critical for query performance. Initial experiments with unpartitioned JSON in Silver exhibited 10-15x slower query performance compared to the final Parquet implementation. Partition granularity tuning (daily in Bronze, monthly in Silver) balanced file count management (avoiding millions of tiny files) with predicate pushdown effectiveness (enabling date-based filtering to skip irrelevant partitions).

NLP and LLM Integration Insights

Prompt engineering emerged as the dominant factor in SQL generation quality, more impactful than model size or capability. Structured prompts with explicit rules, few-shot examples demonstrating correct patterns, and schema metadata injection achieved approximately 85% first-attempt success rate, compared to 40% success for naive prompts lacking these elements. Domain-specific constraints (reserved word handling for “year” and “month”, explicit date range guidance for 2025 vs 2024 queries) addressed systematic errors that general prompting could not resolve.

Multi-layer validation proved necessary for SQL safety in LLM-powered systems. While forbidden keyword filtering provided first-line defense, complexity checks (JOIN limits, subquery depth) prevented resource exhaustion from syntactically valid but pathologically expensive queries. The retry mechanism’s error feedback incorporation substantially improved eventual success rate—approximately 12% of failed first attempts succeeded on second or third attempt after incorporating validation errors into refinement prompts.

Dual-model architecture successfully balanced cost and quality. Gemini 2.0 Flash’s sub-second latency and low per-token cost made interactive SQL generation economically sustainable, while Gemini 2.5 Pro’s advanced reasoning capabilities justified higher costs for analytics where sophisticated insight generation provided user value. Total LLM API costs for development and testing (approximately 1M tokens over 3 months) remained under \$15, demonstrating affordability for academic and small-scale deployment.

System Design Insights

Containerization with Docker Compose dramatically simplified deployment and environment replication. The ability to specify complete infrastructure (MinIO, Dremio, chatbot API, scheduler, frontend) in a single declarative YAML file reduced deployment time from hours (manual service installation and configuration) to minutes (docker compose up). Health checks and restart policies provided resilience against transient failures without monitoring infrastructure.

Modular architecture with clear separation of concerns (extractors vs. processors vs. aggregators, bot logic vs. LLM service vs. validation) enabled parallel development and incremental testing. The ability to develop and test the chatbot independent of pipeline execution, or to add new data sources without modifying Gold layer logic, reduced development coupling and accelerated iteration speed.

State management for incremental processing proved more complex than anticipated but essential for production viability. Atomic state file updates, crash consistency through temporary-file-then-rename patterns, and per-source state isolation were all necessary to achieve reliable daily automated execution. The investment in robust state management (approximately 15% of development time) paid dividends in operational reliability.

11.4 Project Limitations

Data completeness varies significantly by source and procurement lifecycle stage. Tender announcement data exhibits excellent coverage (>95% for title, buyer, publication date), while award and contract data shows substantially lower completeness (60-70% for supplier names, award amounts). This reflects procurement process reality—not all announced tenders result in awards, and award information is often published separately or not at all. Users must account for this completeness variation when interpreting aggregate statistics.

The system is limited to tender and contract award data, excluding other procurement lifecycle stages such as planning (budget allocation, needs assessment), execution (milestone completion, payment schedules), and performance evaluation (contract completion, quality assessment). Future integration of these lifecycle stages would require additional data sources and schema extensions.

Deduplication employs a conservative exact-match strategy (OCID + source_publication_id) rather than fuzzy matching. The same procurement may appear in multiple sources (e.g., both BASE Portugal and TED) with slight variations in field values, and current deduplication would fail to merge these duplicates. Advanced deduplication using similarity measures (title Levenshtein distance, buyer name matching, publication date proximity) or machine learning classification was considered out of scope.

The NLP chatbot is limited to read-only SELECT queries, preventing users from creating derived tables, updating metadata, or performing administrative operations. This design choice prioritizes safety over flexibility, as enabling write operations would require substantially more complex authorization and validation logic. Additionally, the chatbot currently supports only English-language queries, despite the underlying data containing Portuguese and Albanian content.

As an academic prototype, the system has not undergone production hardening for enterprise-scale deployment. Security auditing, load testing beyond development-scale workloads, comprehensive error recovery testing, and compliance certification (GDPR, data retention policies) were not conducted. Deployment in production environments would require these additional validation activities.

11.5 Future Work

Additional data source integration represents the most straightforward extension path. Procurement platforms in France (PLACE), Netherlands (TenderNed), and other EU member states could expand geographic coverage. National platforms in non-EU European countries (Norway, Switzerland) would provide additional perspective on public procurement beyond EU regulatory frameworks.

Advanced deduplication using fuzzy matching or machine learning classification could substantially improve data quality by merging duplicate records from multiple sources. Techniques such as locality-sensitive hashing for scalable similarity search, supervised classification models trained on labeled duplicate pairs, or unsupervised clustering of similar tenders would reduce redundancy and improve aggregate statistics accuracy.

Real-time data ingestion through streaming architectures (Apache Kafka, AWS Kinesis) would enable near-instantaneous data availability compared to the current daily batch processing. This would require architectural evolution from batch-oriented extraction to change data capture patterns, with Bronze layer becoming an append-only event log rather than file-based storage.

Advanced analytics including predictive modeling (forecasting tender volumes, estimating award probabilities), anomaly detection (identifying unusual procurement patterns indicative of fraud or inefficiency), and network analysis (buyer-supplier relationship mapping, procurement market concentration analysis) would extract additional value from the integrated dataset. These capabilities could leverage machine learning frameworks (scikit-learn, PyTorch) integrated into the Gold layer processing pipeline.

Multi-language NLP support enabling queries in Portuguese, Spanish, German, and other European languages would improve accessibility for non-English-speaking users. This would require multilingual LLM models or translation layers, with additional complexity in interpreting queries that mix languages (e.g., English question about Portuguese procurement categories).

Frontend visualization dashboard development would provide graphical interfaces for non-technical users, eliminating the need to interact with API endpoints directly. Technologies such as React, Plotly Dash, or Streamlit could provide interactive visualizations, filters, and drill-down capabilities complementing the chatbot’s natural language interface.

Production deployment on cloud-native infrastructure (Kubernetes orchestration, managed database services, serverless functions for processing) would improve scalability, reliability, and operational efficiency. Migration from Docker Compose to Kubernetes would enable horizontal scaling, automated failover, and integration with enterprise monitoring and logging systems (Prometheus, Grafana, ELK stack).

11.6 Final Remarks

This project demonstrates the feasibility and value of unified European procurement data access through modern data engineering and artificial intelligence techniques. By combining medallion architecture data lakes, distributed query engines, columnar storage formats, and large language models, the system achieves a level of accessibility and analytical capability previously unavailable for cross-border procurement intelligence.

The technical architecture successfully balances multiple competing concerns: storage efficiency vs. query performance (resolved through columnar Parquet format), data quality vs. data volume (resolved through quality metadata flags), system security vs. user flexibility (resolved through multi-layer validation), and cost efficiency vs. analytical capability (resolved through dual-model LLM architecture). These design decisions establish patterns applicable to other domains requiring secure, scalable, user-friendly data platforms.

The integration of natural language interfaces with structured data analytics represents a significant step toward democratizing data access. By eliminating SQL proficiency as a prerequisite for procurement data exploration, the system expands the potential user base from database specialists to business analysts, policy researchers, and SME procurement professionals. This democratization has implications for procurement market transparency, evidence-based policymaking, and equitable access to tender opportunities.

All academic objectives were fully achieved, with the system exceeding minimum requirements in source count (4 vs. 3), record volume (697K vs. minimum unspecified), and feature completeness (chatbot and automation were stretch goals, both delivered). The project validates the medallion architecture pattern for regulated domains, demonstrates safe LLM integration for database querying, and establishes a foundation for future research in public procurement analytics.

12. REFERENCES

Academic and Technical Literature

Databricks. “Medallion Architecture”. *Databricks Glossary*, 2023. <https://www.databricks.com/glossary/medallion-architecture>. Accessed December 2025.

Armbrust, M., et al. “Lakehouse: A New Generation of Open Platforms that Unify Data Warehousing and Advanced Analytics”. *CIDR 2021*, January 2021.

Standards and Specifications

Open Contracting Partnership. “Open Contracting Data Standard Documentation”. Version 1.1, 2023. <https://standard.open-contracting.org/>. Accessed November 2025.

Apache Software Foundation. “Apache Parquet Documentation”. Version 2.0, 2024. <https://parquet.apache.org/docs/>. Accessed November 2025.

Internet Engineering Task Force (IETF). “JSON Schema: A Media Type for Describing JSON Documents”. RFC 8927, December 2020.

International Organization for Standardization. “ISO 8601:2019 - Date and time format”. December 2019.

Technologies and Tools

MinIO, Inc. “MinIO Object Storage Documentation”. <https://min.io/docs/>. Accessed December 2025.

Dremio Corporation. “Dremio Software Documentation”. <https://docs.dremio.com/>. Accessed December 2025.

Ramírez, Sebastián. “FastAPI Framework Documentation”. Version 0.109.0, 2024. <https://fastapi.tiangolo.com/>. Accessed November 2025.

Google LLC. “Gemini API Documentation”. <https://ai.google.dev/docs>. Accessed December 2025.

The pandas development team. “pandas: Powerful Python Data Analysis Toolkit”. Version 2.x, 2024. <https://pandas.pydata.org/docs/>. Accessed November 2025.

Apache Software Foundation. “Apache Arrow Python (PyArrow) Documentation”. Version 14.x, 2024. Accessed November 2025.

Docker, Inc. “Docker Documentation”. <https://docs.docker.com/>. Accessed November 2025.

Moessner, G. “sqlparse: Non-validating SQL parser for Python”. Version 0.4.4, 2023. <https://github.com/andialbrecht/sqlparse>. Accessed December 2025.

Data Sources

Open Contracting Partnership. “OCDS Data Portal”. <https://data.open-contracting.org/>. Accessed continuously September-December 2025.

Agência para a Modernização Administrativa, IP. “dados.gov.pt - BASE Portal OCDS Dataset”. <https://dados.gov.pt/pt/datasets/ocds-portal-base-www-base-gov-pt/>. Accessed continuously September-December 2025.

IMPIC - Instituto dos Mercados Públicos, do Imobiliário e da Construção. “BASE - Portal de Contratação Pública”. <https://www.base.gov.pt/>. Accessed November 2025.

Publications Office of the European Union. “TED (Tenders Electronic Daily)”. <https://ted.europa.eu/>. Accessed November 2025.

Research and Methodology

Google AI. “Prompt Engineering Best Practices”. https://ai.google.dev/docs/prompt_best_practices. Accessed December 2025.

Brown, T., et al. “Language Models are Few-Shot Learners”. *Advances in Neural Information Processing Systems*, vol. 33, 2020, pp. 1877-1901.

OWASP Foundation. “SQL Injection Prevention Cheat Sheet”. [https://cheatsheetseries.owasp.org/cheatsheets/SQL_Injection](https://cheatsheetseries.owasp.org/cheatsheets/SQL_Injection_Prevention_Cheat_Sheet.html). Accessed November 2025.

European Commission. “European Public Procurement: Study on Administrative Capacity in the EU”. Publications Office of the European Union, 2022.
